

Distorsión de lente

Visión artificial

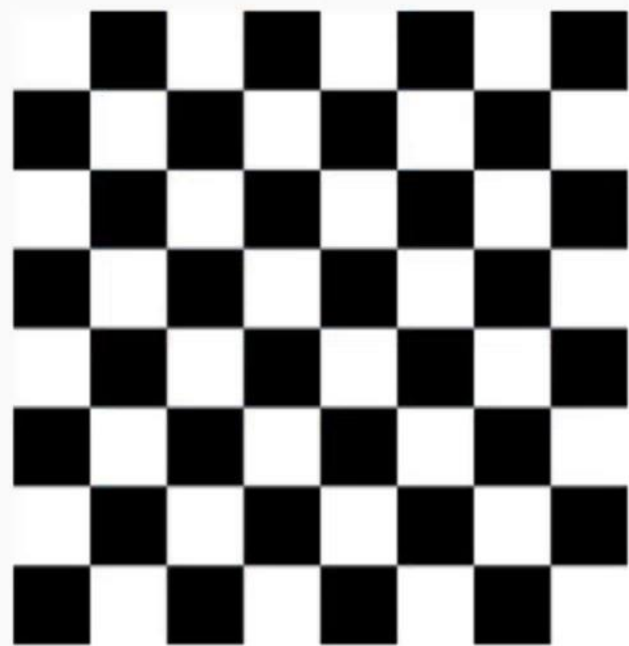
Alejandro Silvestri

Distorsión

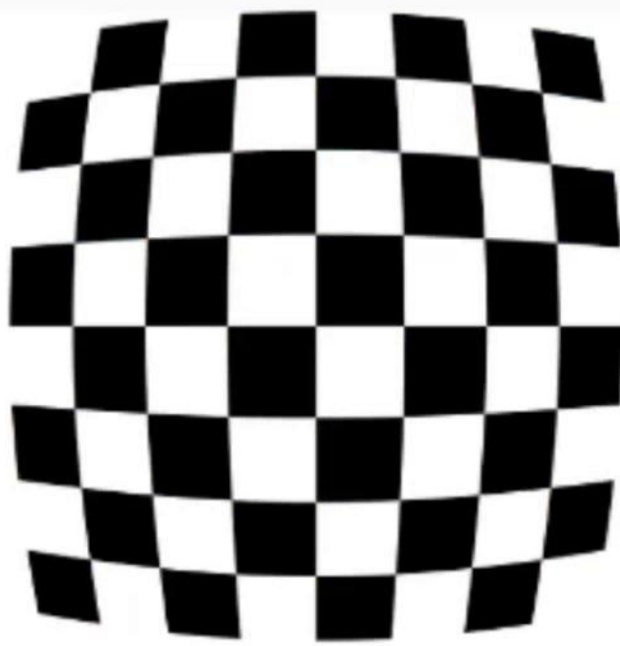


- Son apartamientos respecto de la colineación
 - Ocasionados por defectos constructivos
 - Lentes de vidrio tienen menos distorsión que lentes de plástico
- Son particulares de cada cámara y lente individual
 - Incluyendo cámaras y lentes de un mismo modelo fabricadas en serie
- Se expresan como coeficientes de un **modelo de distorsión**
- **Antidistorsión (undistortion)**
 - es una transformación geométrica
 - convierte una imagen distorsionada en una colineación

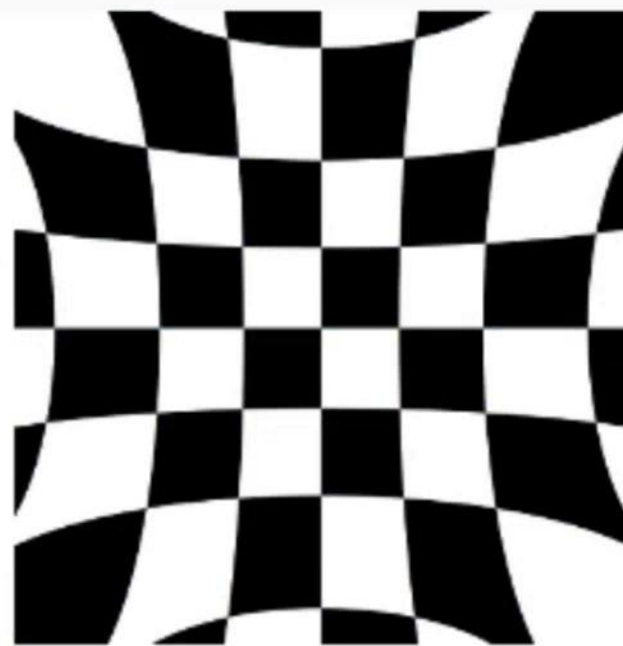
Distorsión radial



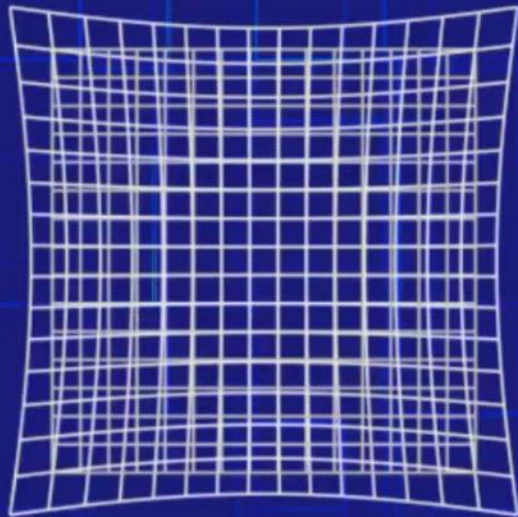
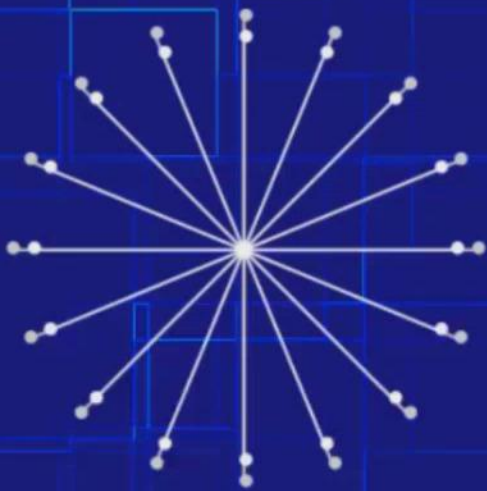
No distortion



Negative radial distortion
(Barrel distortion)



Positive radial distortion
(Pincushion distortion)



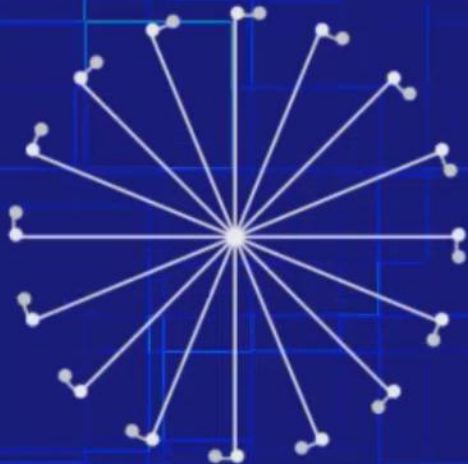
Distorsión radial



- está presente en todas las cámaras comunes
- es necesario compensar esta distorsión
- la distorsión se aproxima con esta serie de Taylor simétrica

$$x_{distorted} = x(1 + k_1r^2 + k_2r^4 + k_3r^6)$$

$$y_{distorted} = y(1 + k_1r^2 + k_2r^4 + k_3r^6)$$

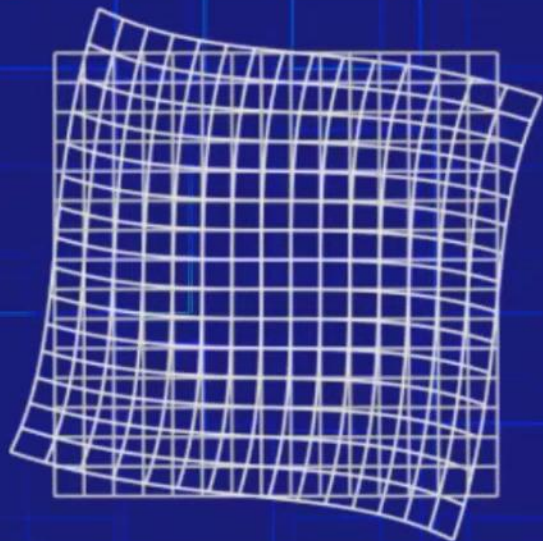


Distorsión tangencial

- causada por inclinación de la lente
- es un fenómeno despreciable en cámaras comunes, no se suele considerar

$$x_{distorted} = x + [2p_1xy + p_2(r^2 + 2x^2)]$$

$$y_{distorted} = y + [p_1(r^2 + 2y^2) + 2p_2xy]$$



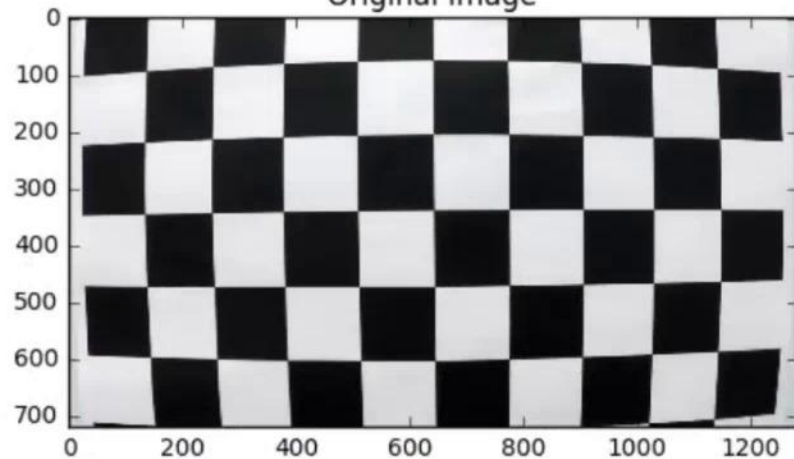
Coeficientes de distorsión

- Coeficientes de distorsión radial **k**
- Coeficientes de distorsión tangencial **p**

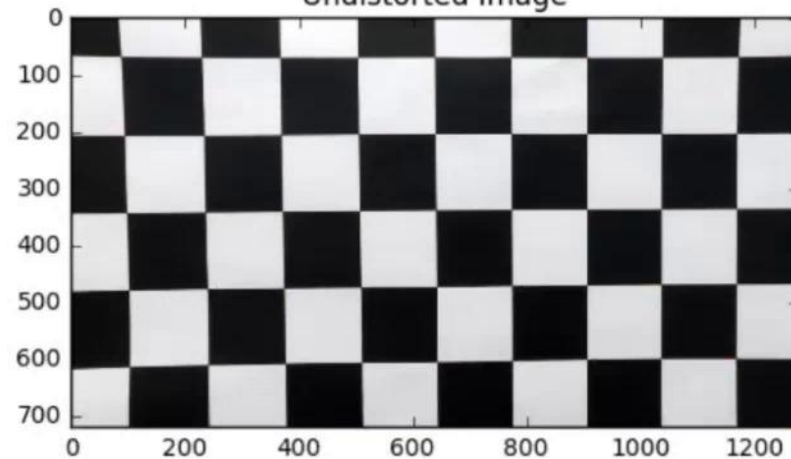
$$\textit{distortion_coefficients} = (k_1 \quad k_2 \quad p_1 \quad p_2 \quad k_3)$$

- Son **cero** si no hay distorsión
- OpenCV admite arrays de varias longitudes:
 - k_1, k_2, p_1, p_2 [, k_3 [, k_4, k_5, k_6]]

Original Image



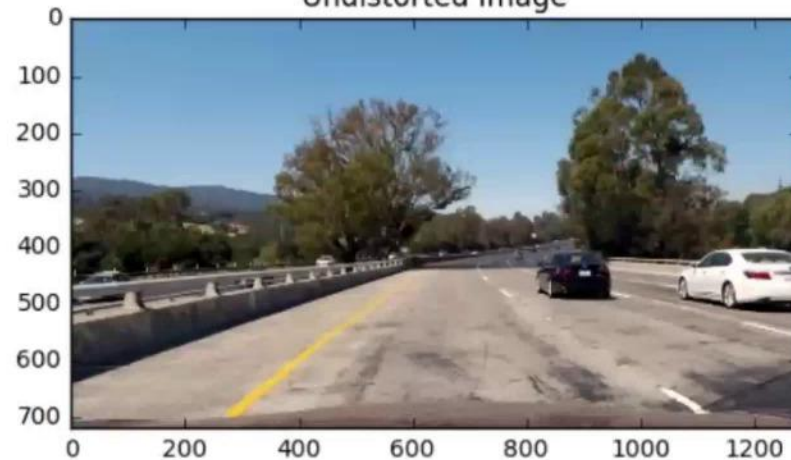
Undistorted Image



Original Image

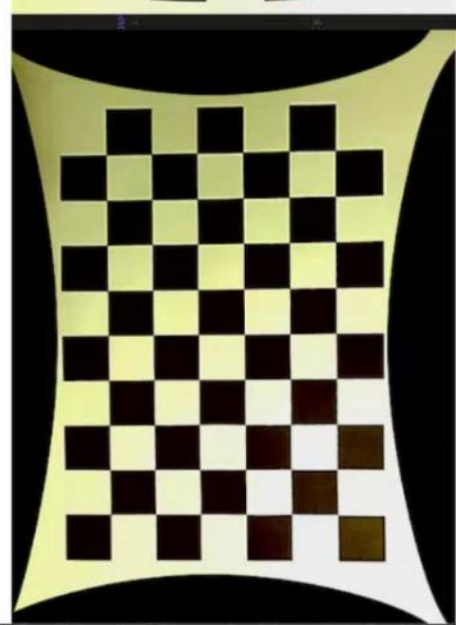
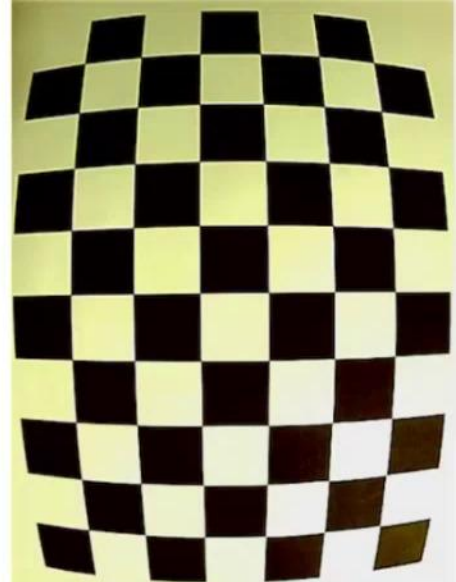
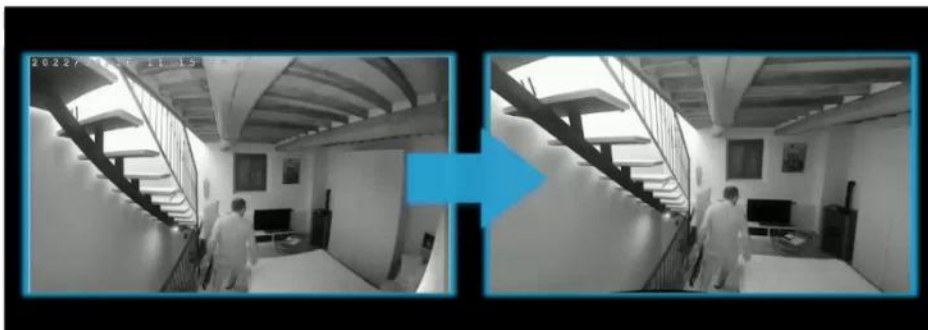


Undistorted Image



Antidistorsión

- Transformación geométrica
 - produce una **colineación**
 - compensando la distorsión



Funciones undistort*

- Requieren
 - Matriz de cámara
 - Coeficientes de distorsión
- **undistort()**
 - **warp**, genera una imagen antidistorsionada
 - para **visualización**
- **undistortPoints()**
 - antidistorsiona **coordenadas de puntos**
 - se usa típicamente sobre coordenadas de **keypoints**



A close-up photograph of a person wearing a light blue long-sleeved shirt, kneeling and using a tire pressure gauge on the front wheel of a red car. The gauge is silver with a black handle and a white face with black markings. The person's hands are visible, one holding the gauge and the other holding a small tool to adjust the valve. The car's body is a vibrant red, and the wheel is a silver alloy. The background is slightly blurred, showing more of the car and some outdoor lighting.

Calibración de cámara

Visión artificial

Alejandro Silvestri

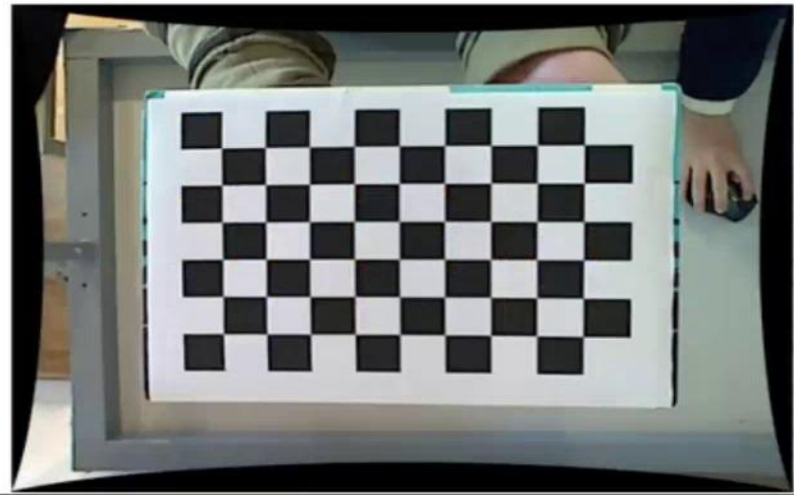
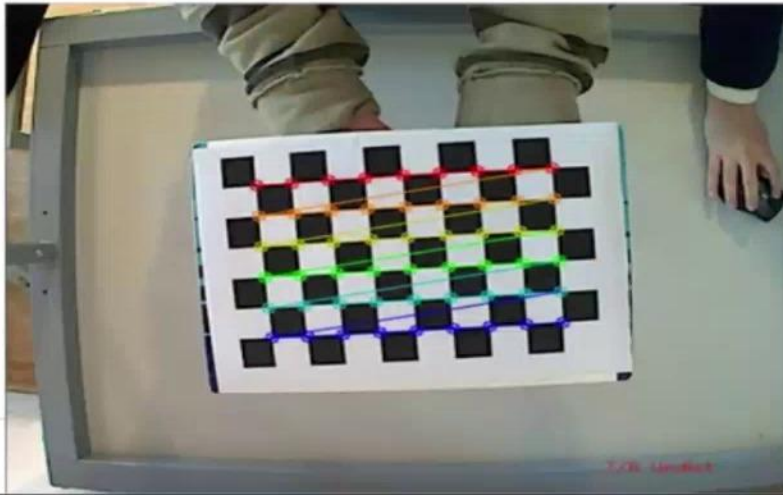
Calibración de cámara

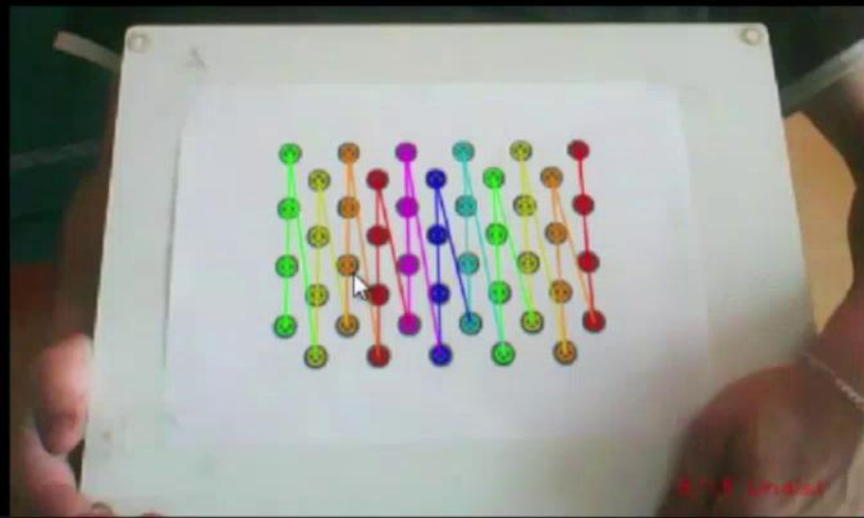
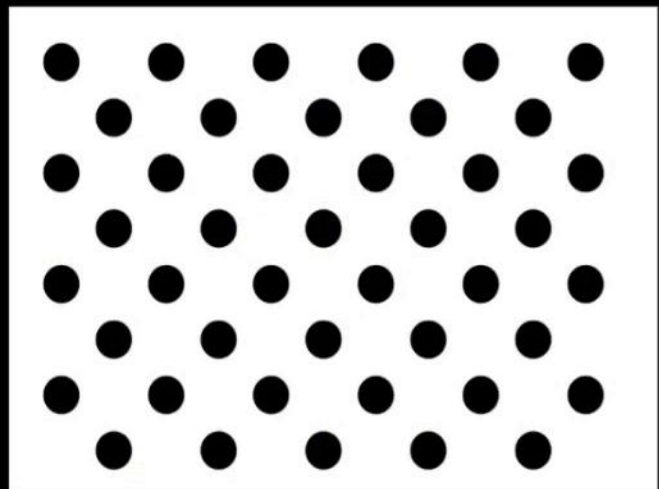
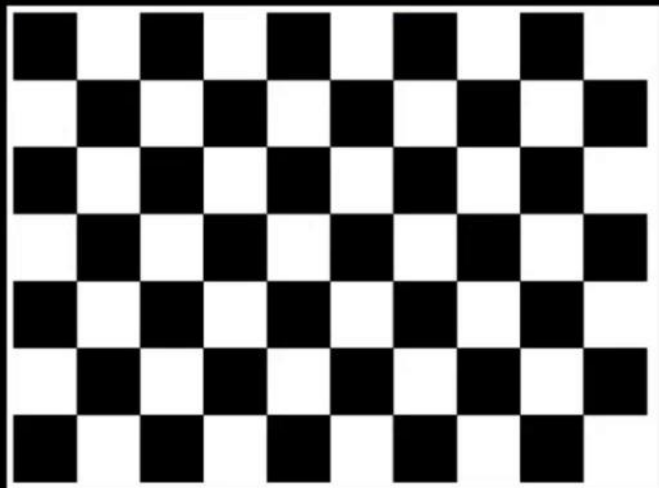


- **No es un ajuste de la cámara**
 - no se altera la cámara
- Es un procedimiento de **medición de parámetros** de la cámara
 - **Parámetros intrínsecos**
 - **Coeficientes de distorsión**
- Estos parámetros varían entre cámaras
 - incluso entre cámaras del mismo modelo
 - el fabricante usualmente no los proporciona, porque requiere determinarlos para cada unidad

Patrones de calibración

- Son imágenes planas que se presentan ante la cámara en diversas posiciones
- Las imágenes tienen patrones de coordenadas conocidas
- Los patrones son fácilmente identificables como keypoints





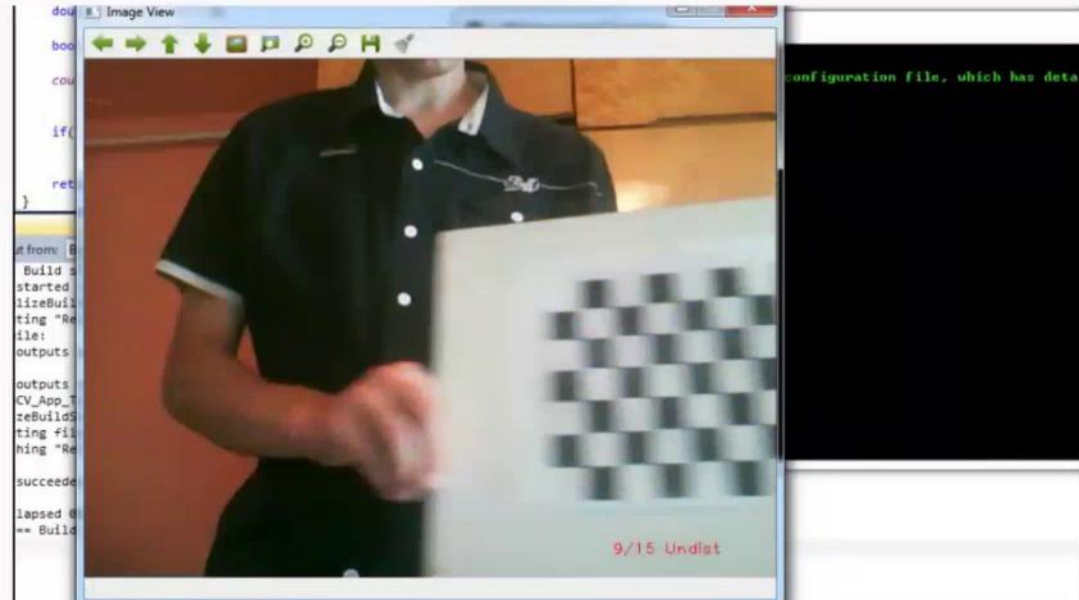
Proceso de calibración

-
- El proceso de calibración tiene como objetivo determinar
 - Matriz de cámara (parámetros intrínsecos)
 - Coeficientes de distorsión de lente
 - La calibración consiste en
 - observar patrones en posiciones diversas
 - 60 tomas
 - extraer keypoints y comparar dónde están y dónde deberían estar si fuera una homografía
 - aplicar un solver para determinar los parámetros que minimizan el error
 - La calibración de cámara **no modifica la cámara**

Proceso de calibración

- Escenas emocionantes

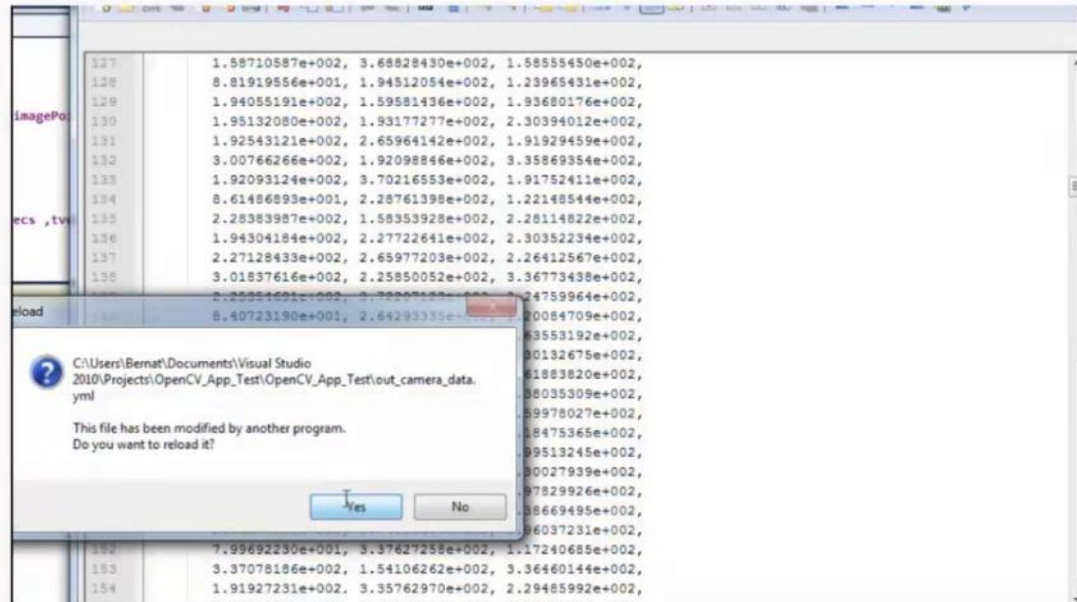
- 0:42
- 2:25
- 3:10



Proceso de calibración

- Escenas emocionantes

- 0:42
- 2:25
- 3:10



Proceso de calibración

- Escenas emocionantes

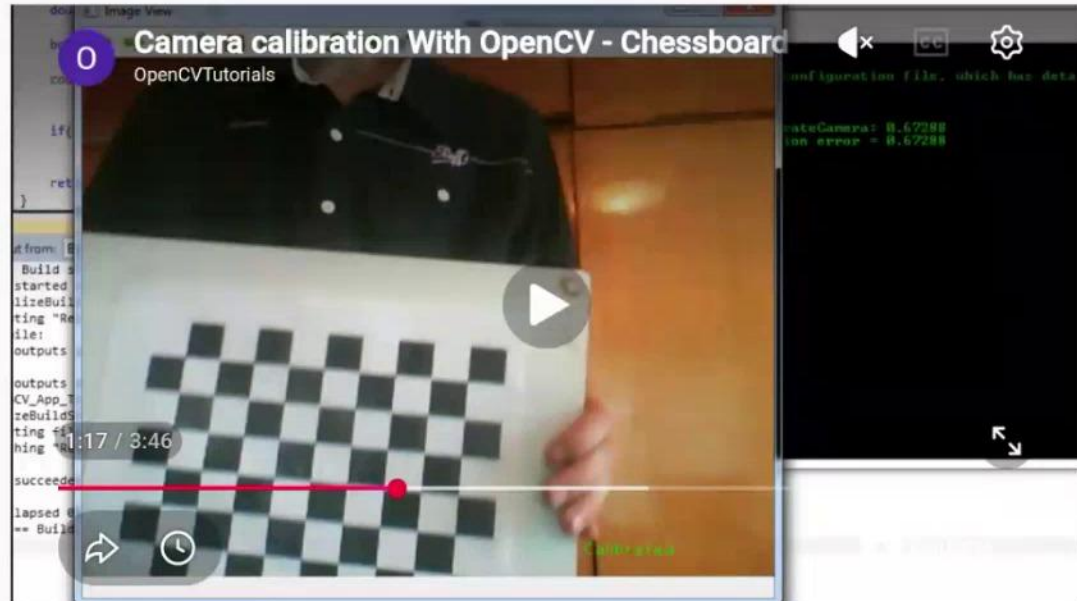
- 0:42
- 2:25
- 3:10

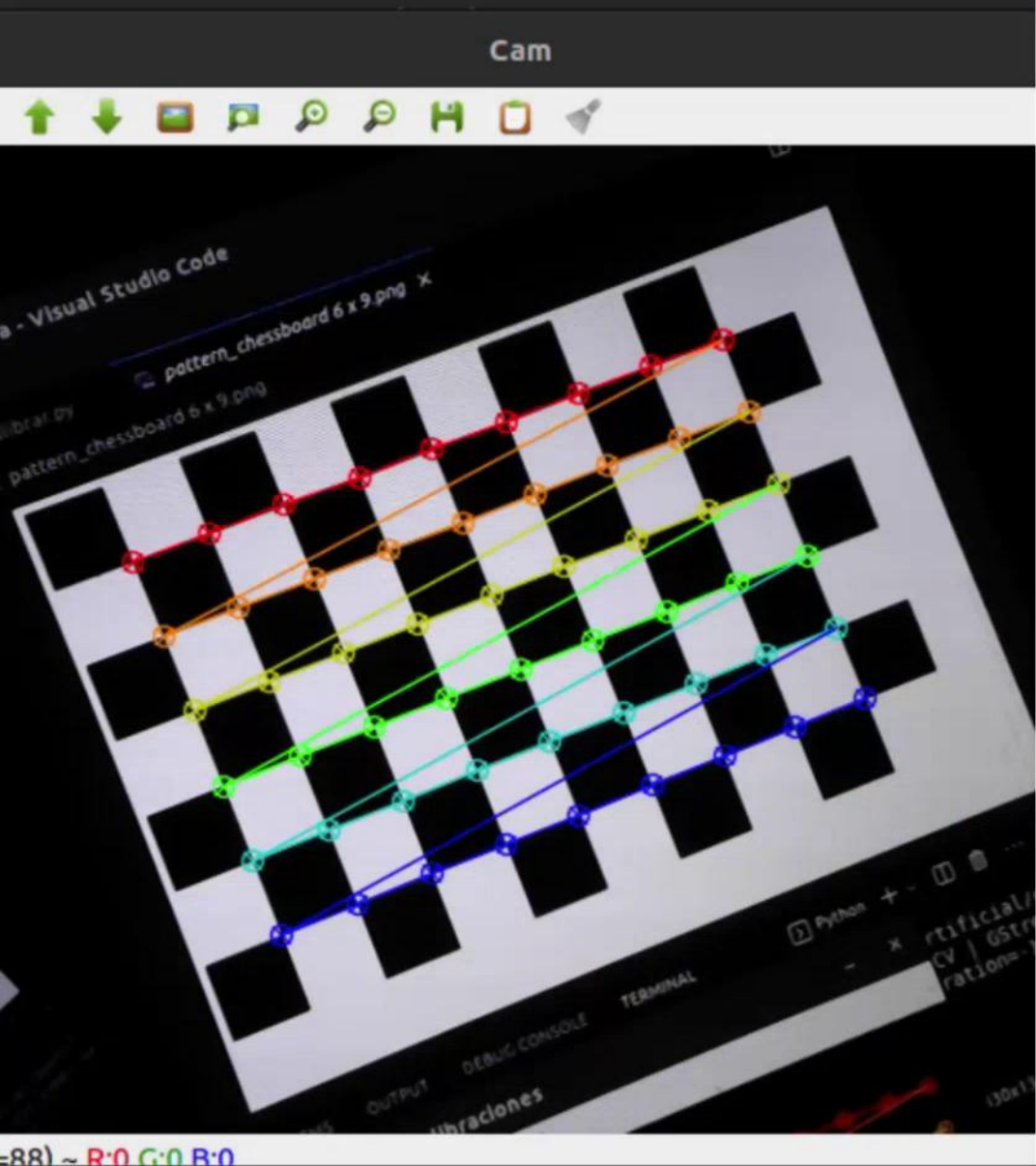


Proceso de calibración

- Escenas emocionantes

- 0:42
- 2:25
- 3:10





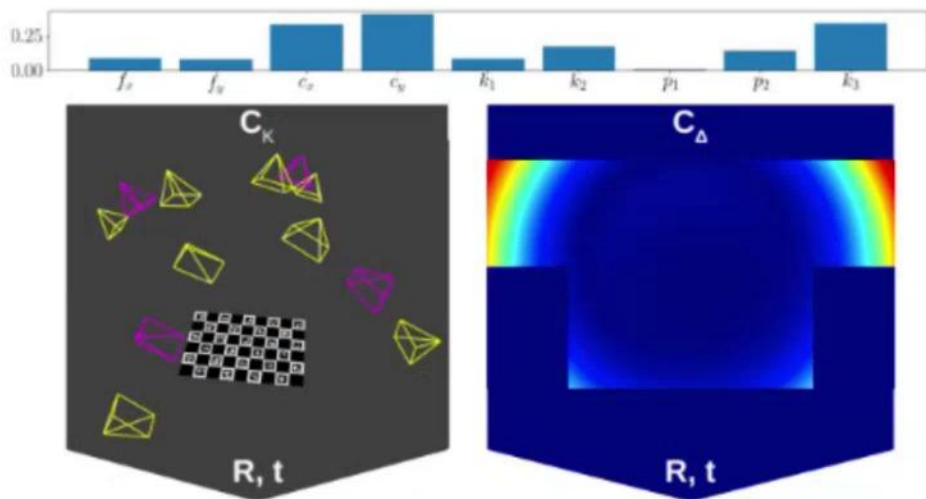
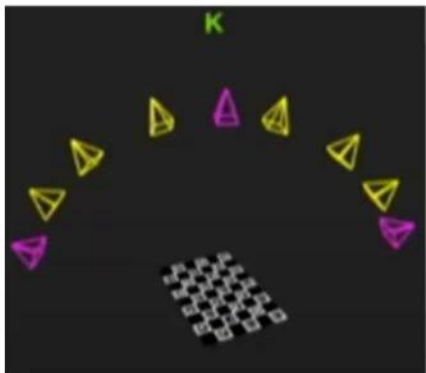
Calibración en Python

- Calibración con webcam
 - [Calibration](#), Python
- Calibración con archivos de imágenes
 - esta [página](#) presenta código Python para calibrar cámaras
 - procesa una secuencias de archivos jpg, con las fotos del patrón de calibración en diferentes poses

Calibrador online

- sitio web estático
- repositorio en GitHub:
<https://github.com/Laboratorio-de-Robotica/camera-calibrator-online>



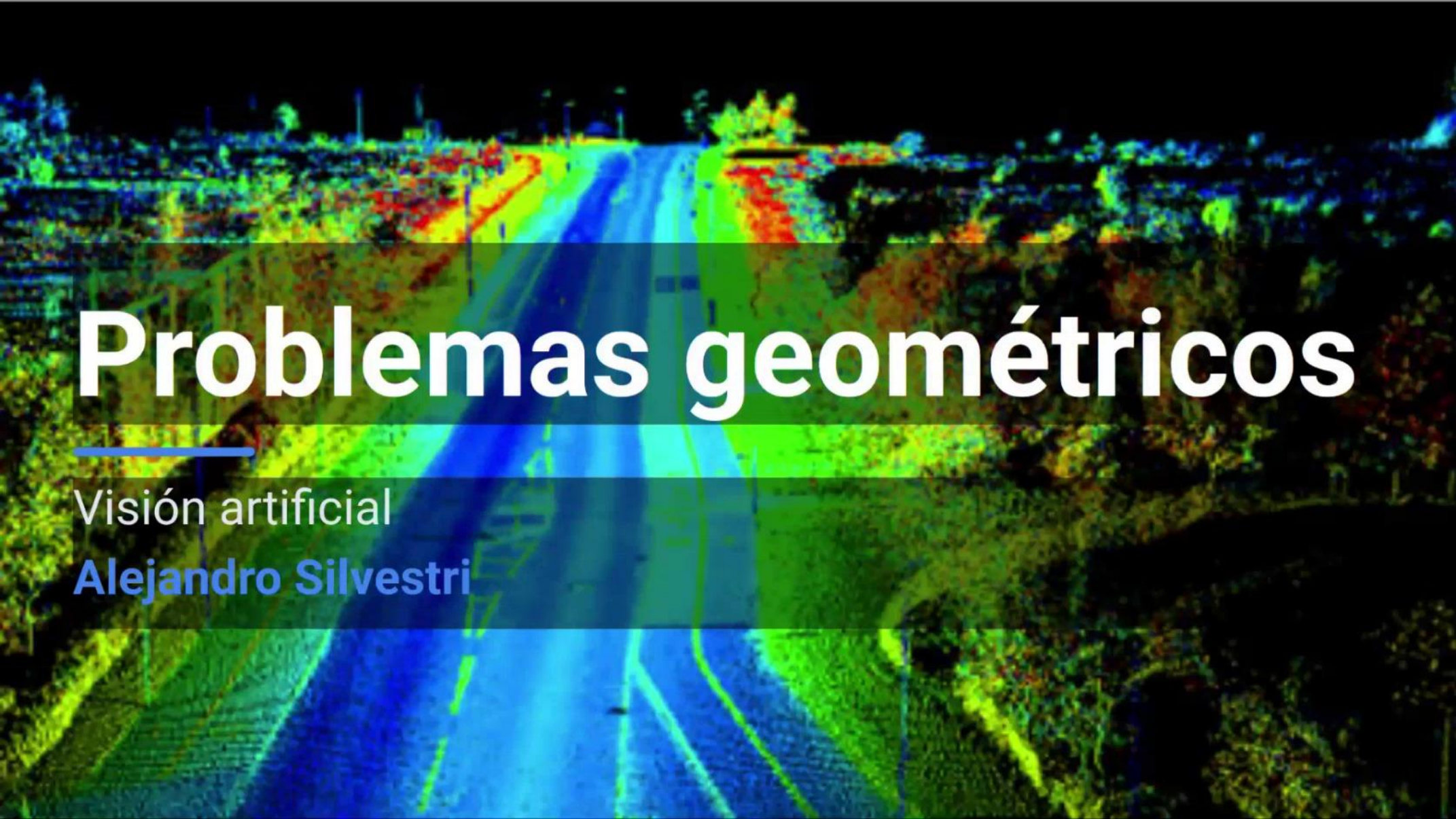


Tomas óptimas

- Arcos circulares
 - horizontal
 - vertical
- Tomas frontales
 - patrón cubriendo la imagen completa
 - patrón en las esquinas
 - superior izquierda
 - superior derecha
- 19 tomas como mínimo

Enlaces

- Referencia
 - [Referencia de calibración pinhole, y teoría](#) (ver Detailed description)
- Tutoriales y ejemplos
 - [Calibration](#) en Python
 - [Calibración](#) en Python en OpenCV
 - [Calibración con teoría](#), en C++
- Documentos
 - [Pinhole camera](#)
 - [Calibración de cámara con Python](#)



Problemas geométricos

Visión artificial

Alejandro Silvestri



Cámaras para visión artificial

Visión artificial
Alejandro Silvestri

Cámaras industriales

- Preparadas para montaje fijo
 - no para sostener en la mano
- Conexión por cable
 - en lugar de batería como las cámaras portátiles
 - cables de
 - alimentación
 - video
 - conectores robustos
 - con rosca
 - pueden ser estancos



Características especiales



- Lente adecuada a la cantidad de luz que debe recibir
 - No es tan relevante cuando el ambiente es controlado
- Zoom y foco fijos
 - sin son ajustables, tienen fijaciones para que no se muevan luego de configurarlos
- Fps
 - aplicaciones industriales pueden requerir tasas elevadas de cuadros por segundo, muy por encima de los convencionales 30 fps
- Espectro ampliado
 - infrarrojo
 - ultravioleta
 - cámaras térmicas
 - usualmente de muy baja resolución

Tipos de sistemas para visión artificial

1. Cámara de visión artificial integrada
 2. Equipo de visión artificial, con conexión para cámara remota
 3. Sistema de visión artificial en PC servidor
 4. Sistema en la nube
- Estos sistemas suelen ser cerrados
 - Tienen su propia interfaz de programación, usualmente programación gráfica
 - No admiten complementarse con código abierto
 - Los algoritmos que no tienen se pueden comprar sólo si el proveedor los tiene disponibles

Cámara de visión artificial integrada



- Son más voluminosas que las cámaras industriales genéricas
- Incluyen hardware para procesar visión artificial en tiempo real
 - tienen baja capacidad de cómputo
 - se usan algoritmos livianos
 - sólo usan algoritmos pesados cuando pueden dedicarle mucho tiempo
- Puerto de datos para programar y comunicar
- Puertos de entrada y salida ON/OFF tipo PLC
 - de este modo se puede integrar a una estación automática como un sensor más, o incluso como un controlador

Equipo de visión artificial

- Computadora propietaria en una caja cerrada
- Mayor capacidad de procesamiento que una cámara integrada
- Se monta en sitio, cerca de la cámara industrial
- Funciones similares a la cámara integrada
 - mayor gama de algoritmos, especialmente los que requieren mayor procesamiento



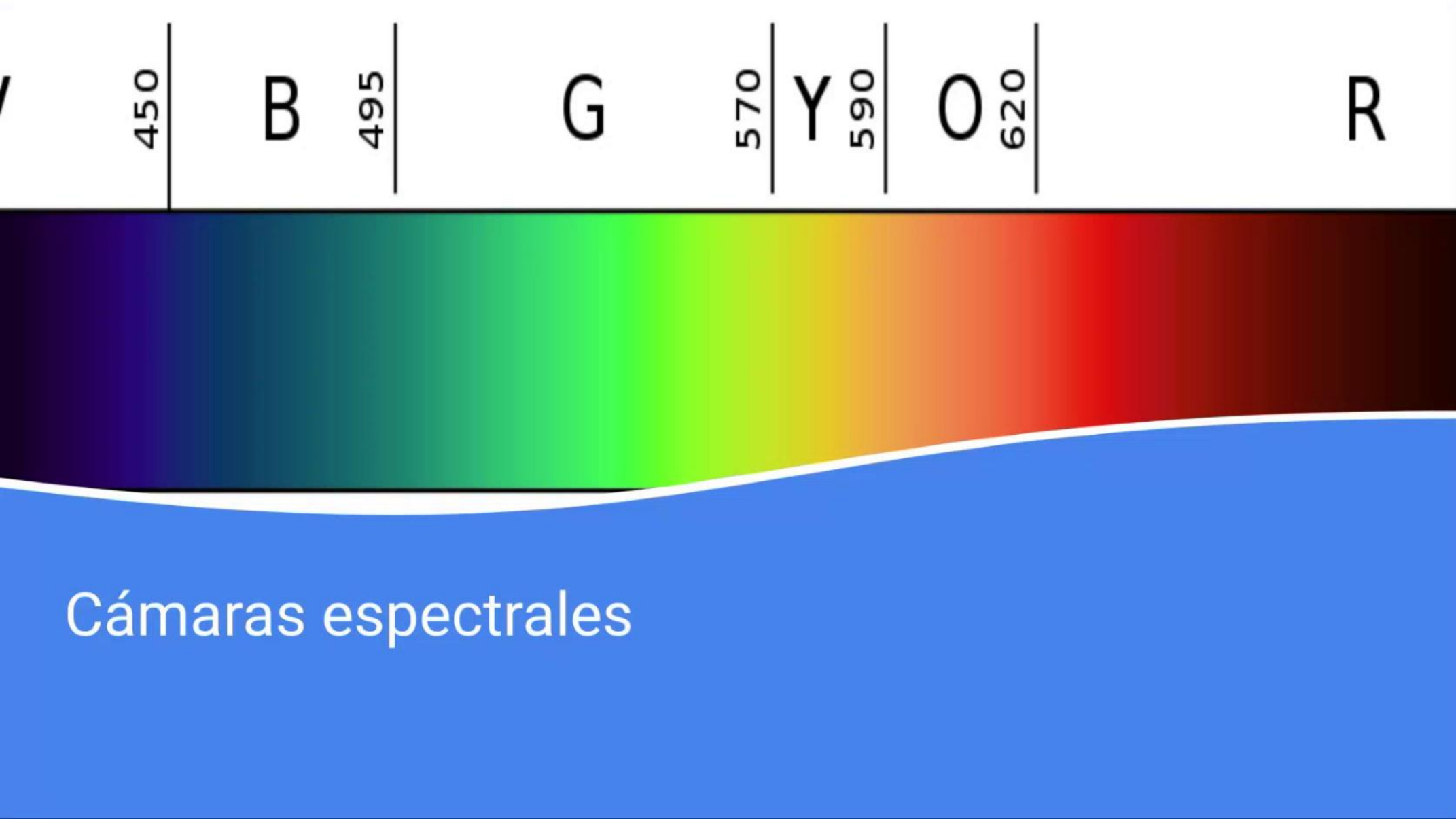
Sistema de visión artificial en servidor

- El sistema es un software que se instala en servidores
 - el cliente elige PC tan potente como quiera
- Es un sistema central y con gran poder de cómputo
 - no están en sitio, sino en datacenter
- Las cámaras envían imágenes por red



Visión artificial en la nube: Industria 4.0

- Proveedores principales de computación en la nube
 - AWS: Amazon Web Services
 - Microsoft Azure
 - Google Cloud
 - el cliente compra capacidad de procesamiento
- Están en la vanguardia de visión artificial con DNN
 - Google Cloud se destaca por su disponibilidad de TPU, procesadores potentes y específicos para DNN
- Las imágenes se suben a la nube para su procesamiento
 - No es un proceso instantáneo
 - Tiempo de subida
 - Tiempo de procesamiento



V

450

B

495

G

570

Y

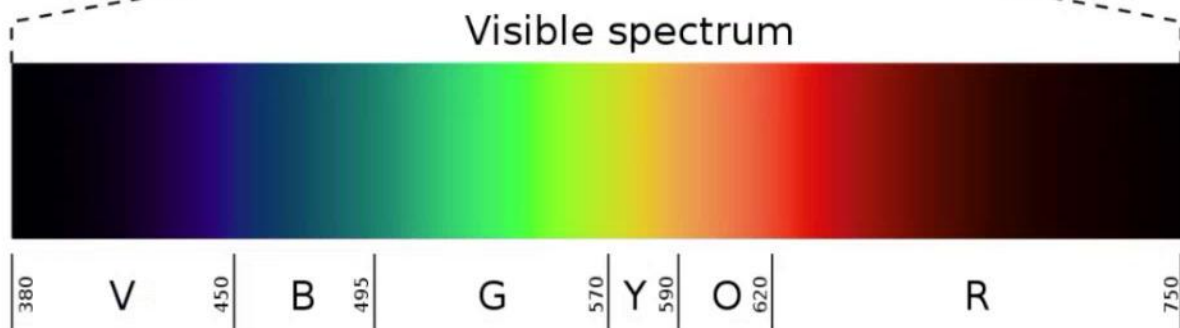
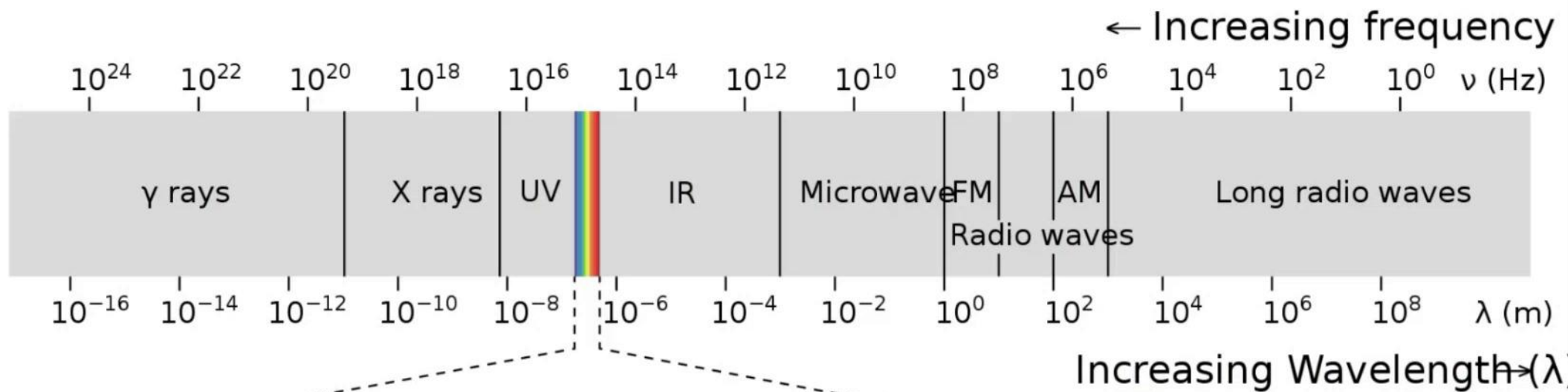
590

O

620

R

Cámaras espectrales



Espectros

- Cada píxel registra la intensidad de radiación que recibe
- Esta radiación está limitada por un filtro pasa banda
- Se produce una imagen monocromática para cada espectro
- Una cámara monocromática registra el espectro visible
- Una cámara color es multiespectral, registra 3 imágenes monocromáticas, cada una sintonizada en una banda de color: R, G y B





Cámaras espectrales

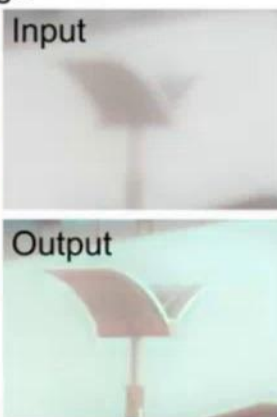
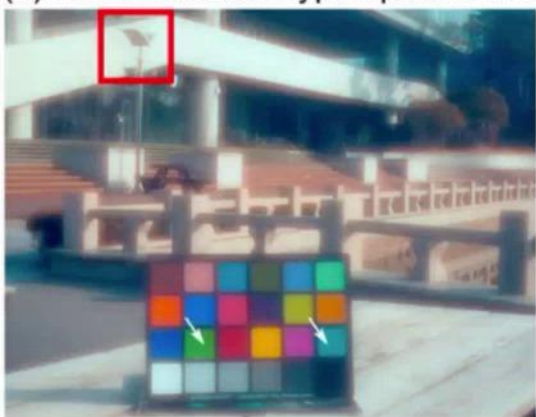
- Cámaras monocromáticas sintonizadas en bandas diferentes a la de la luz visible
- Rangos espectrales
 - Infrarrojas ()
 - ultravioletas (750 THz a 30 PHz)
 - rayos X (30 PHz a 30 EHz)
- Cámara térmica
 - thermal camera
 - cámara infrarroja en la banda de radiación de cuerpo negro para temperatura ambiente



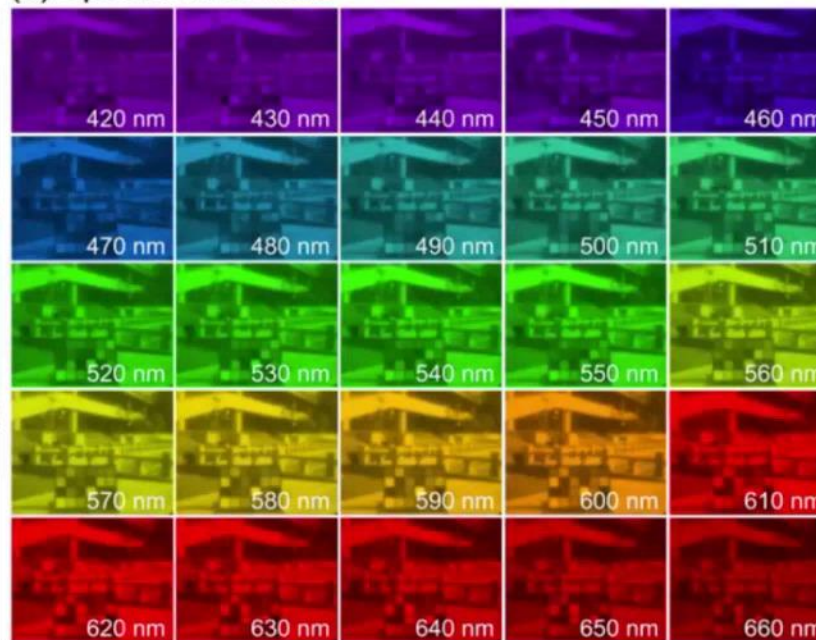
Cámaras multispectrales

- Registran imágenes monocromáticas para varios espectros
 - en rigor, una cámara color común es multispectral
 - se reserva el nombre multispectral para cámaras sintonizadas en bandas diferentes a la de la luz visible

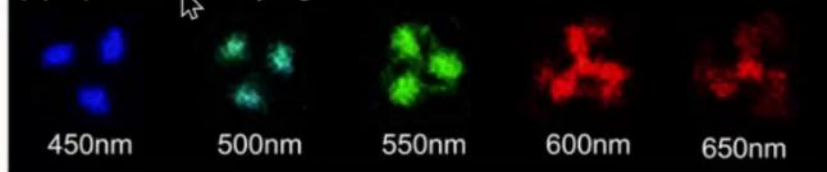
(b) Reconstructed hyperspectral image



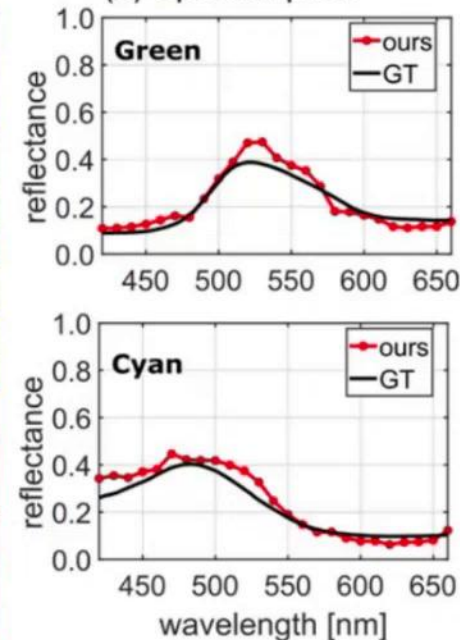
(d) Spectral channels



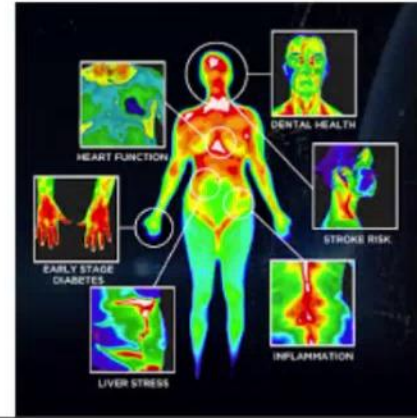
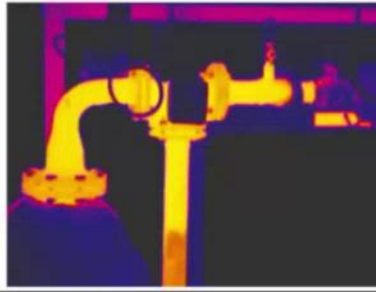
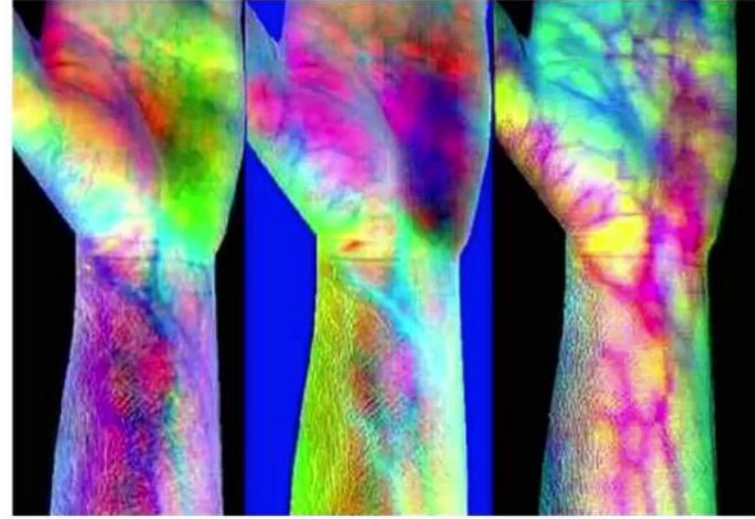
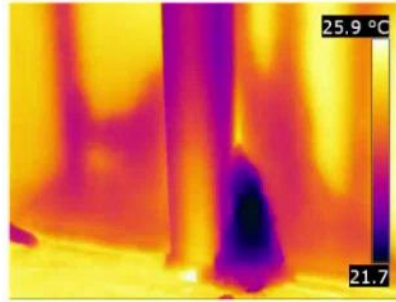
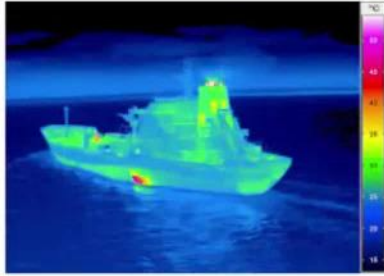
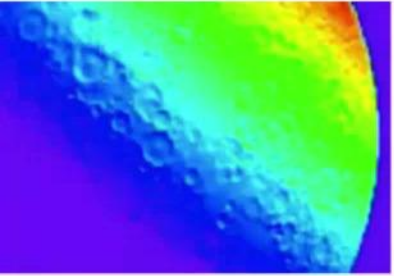
(c) Spectrally-varying PSFs



(e) Spectral plots

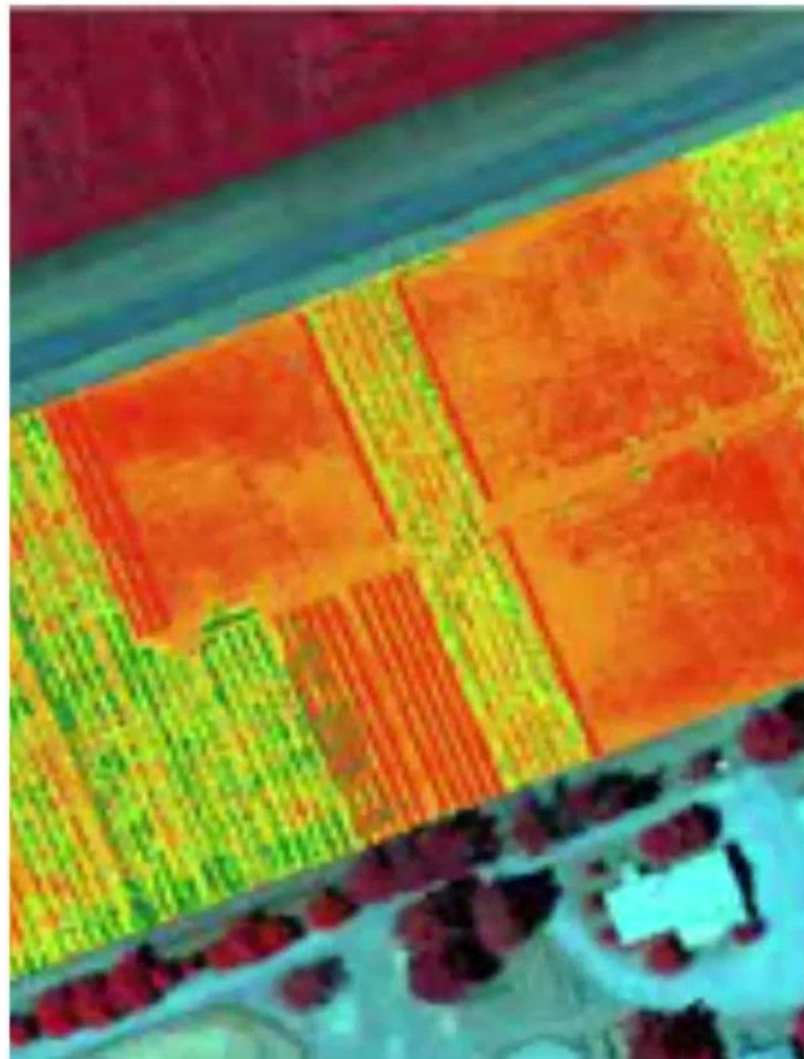


Aplicaciones



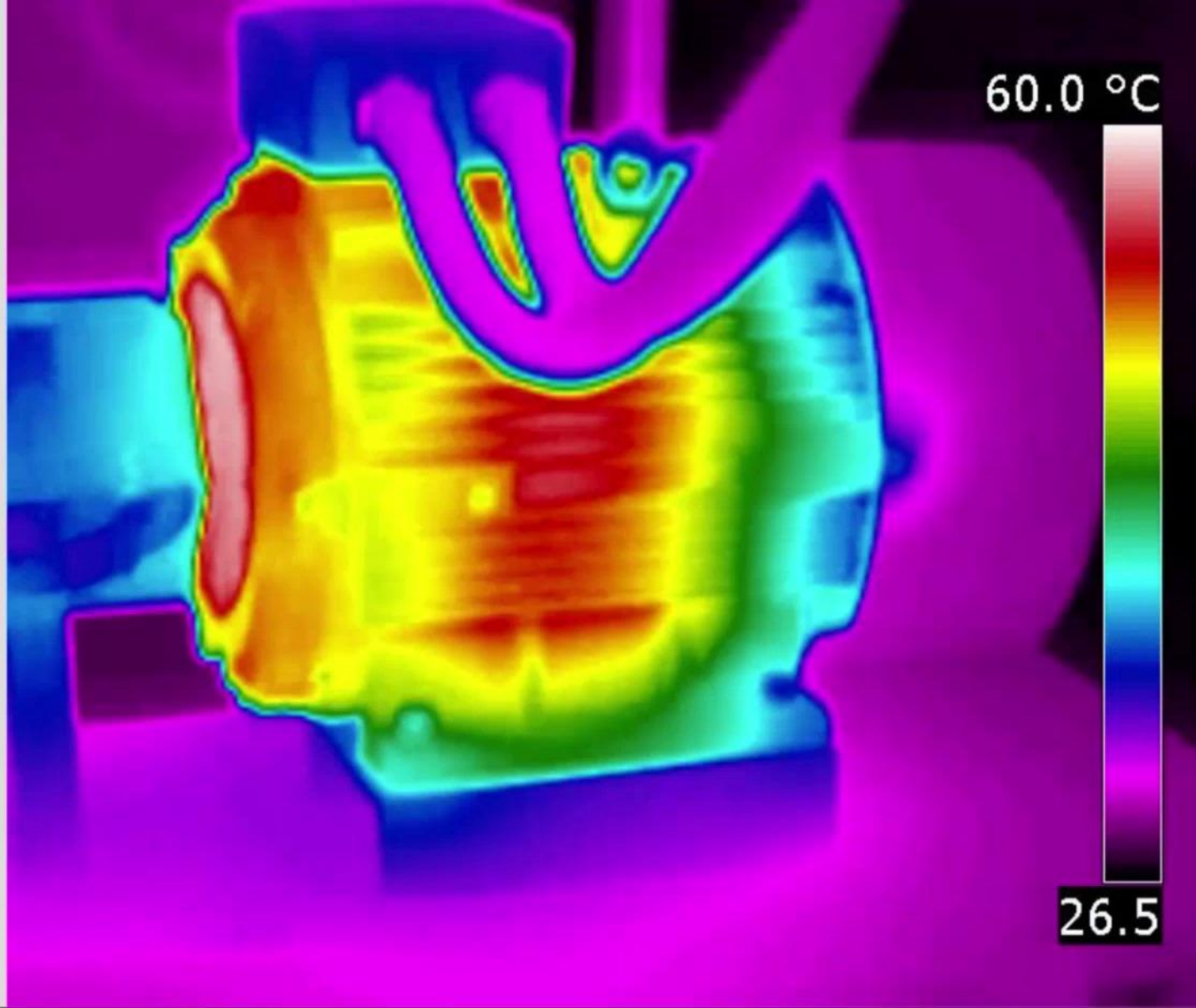
Cámaras de temperatura

- Radiación de cuerpo negro
 - la materia emite radiación con un espectro de frecuencias proporcionales a la temperatura
- Cámara multispectral en el infrarrojo lejano
 - Emisión de cuerpo humano: $7\mu\text{m}$ a $14\mu\text{m}$



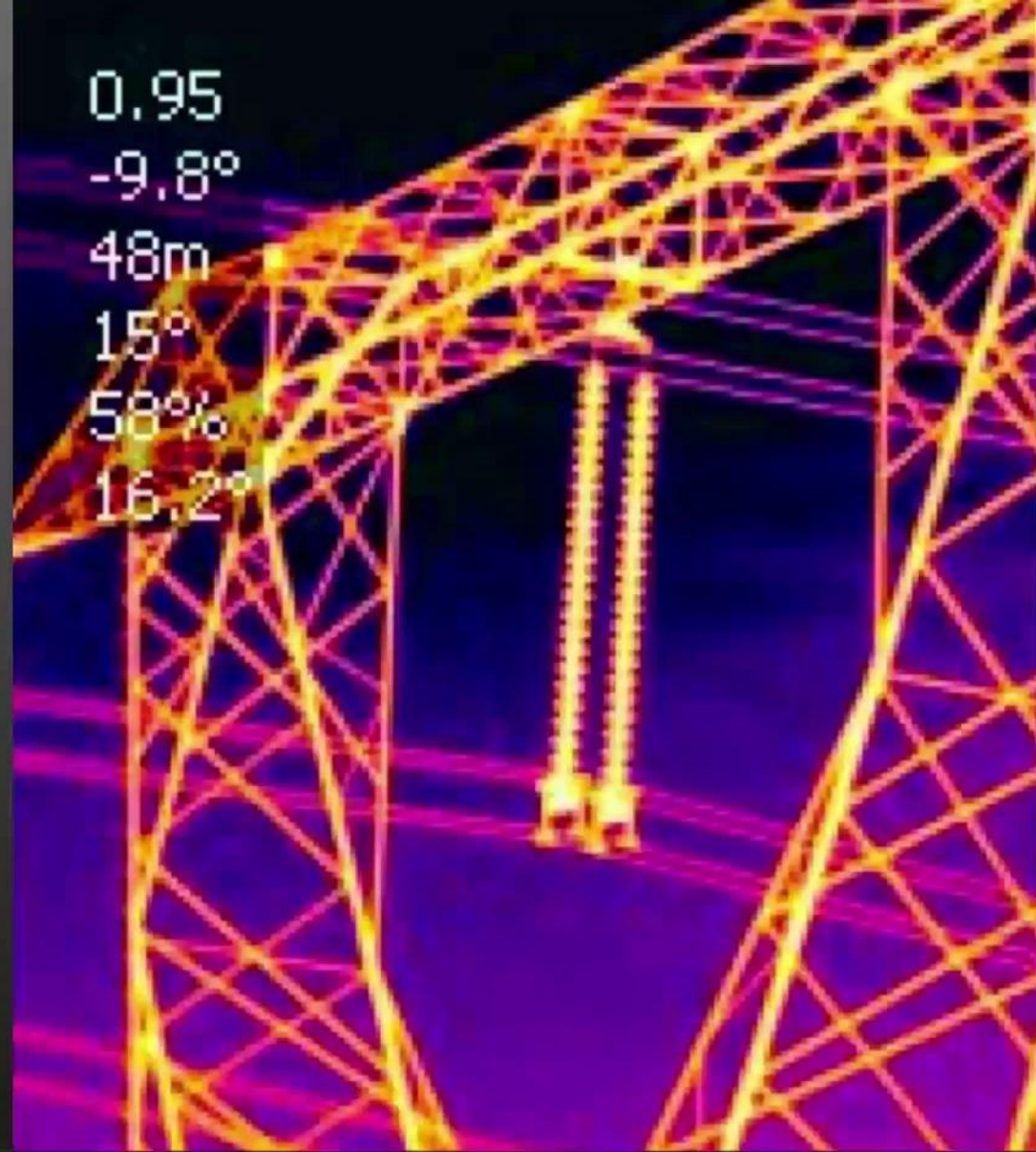
Visualización

- Se aplica un mapa de color que representa la temperatura
- La escala de colores no está normalizada
- Algunas escalas comienzan en azul (frío) y terminan en rojo (caliente), y negro para indicar que no hay lectura



Limitaciones de las cámaras térmicas

- Otras radiaciones (las que no son de cuerpo negro) interfieren la lectura de la temperatura
 - mostrando temperaturas inusualmente altas
- La temperatura no es medida directamente
 - es deducida a partir de la lectura multiespectral para cada píxel
 - no interesa la intensidad de la radiación, sino la proporción entre las bandas
 - radiaciones débiles (afectadas por la distancia) producen equivocadamente lecturas de menor temperatura





Cámaras sintéticas

Imágenes sintéticas

- No se obtienen por proyección de radiación
- Las imágenes se generan artificialmente
 - Se componen mediciones de sensores
 - Ecografías
 - Tomografías

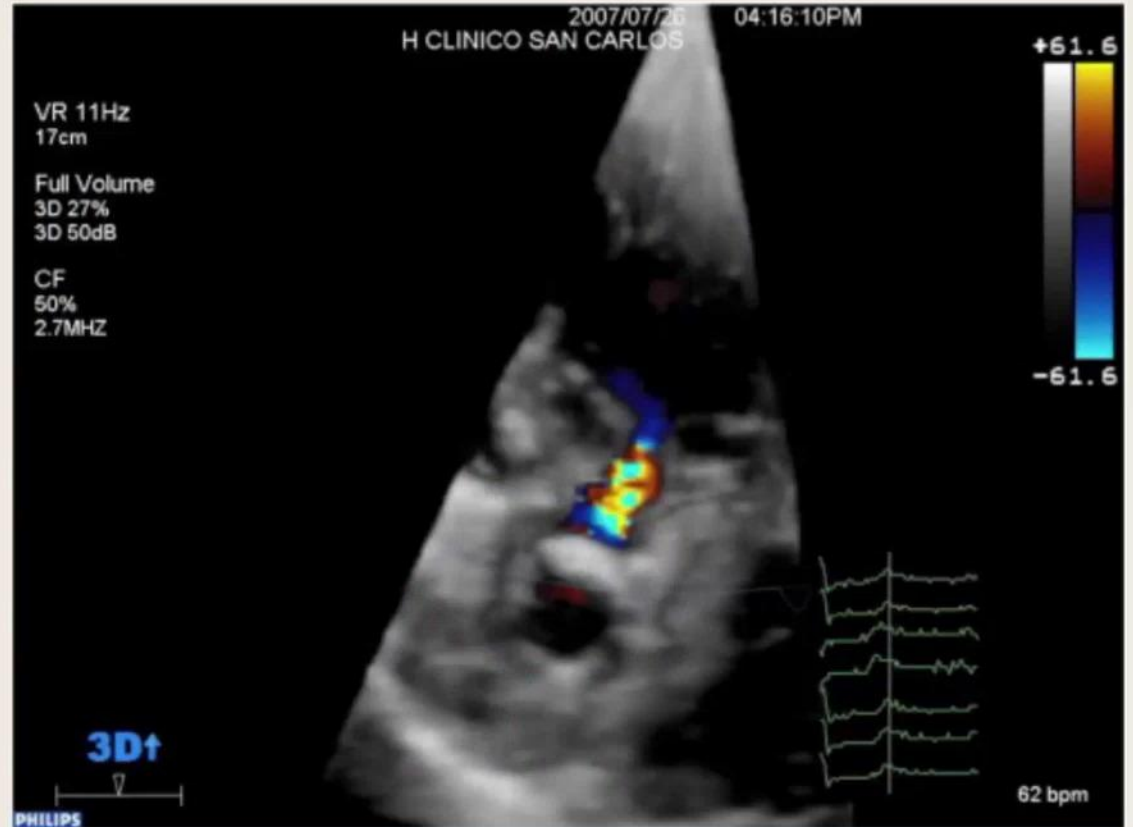
Ecografía

- Produce un rayo de ultrasonido en una dirección
- Registra el eco (función del tiempo)
 - se producen donde cambia la densidad
 - la velocidad del sonido es el factor para convertir tiempo en distancia
- Matriz de eco
 - una fila para cada dirección (para cada ángulo)
 - el eco se desarrolla en el eje X (tiempo)
- Imagen
 - monocromática
 - warp anti polar
- Escala
 - velocidad del sonido para la profundidad
 - ángulo del radio informado por el sensor



Eco Doppler

- Sensores más precisos pueden medir el cambio de frecuencia del eco
 - una medición indirecta de la velocidad de desplazamiento radial del objeto que produce el eco
- Se visualiza con una escala de color sobre una imagen monocromática



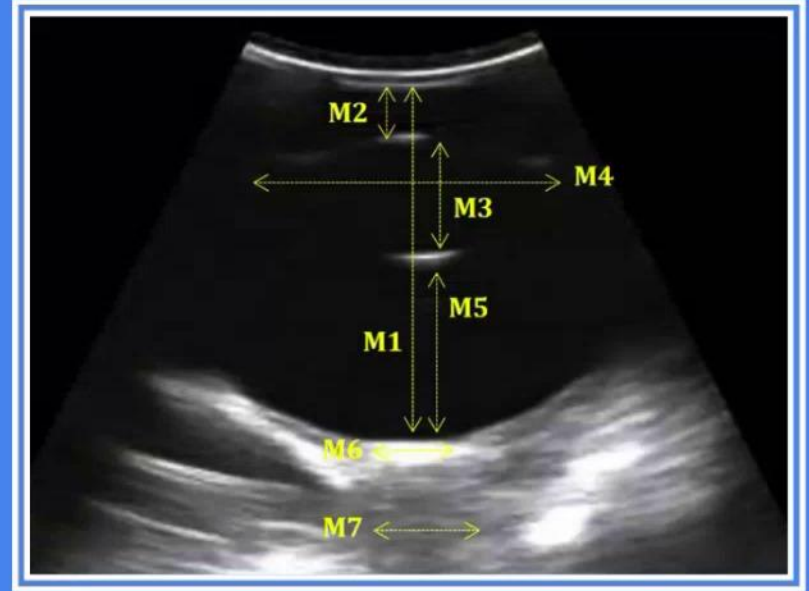
Cámaras de profundidad



90 mm x 25 mm x 25 mm

Métricas

- Las imágenes representan superficies reales
- Se registran coordenadas de puntos y se miden distancias entre puntos
- La escala de píxeles a mm es indirecta, requiere una correcta calibración del sensor



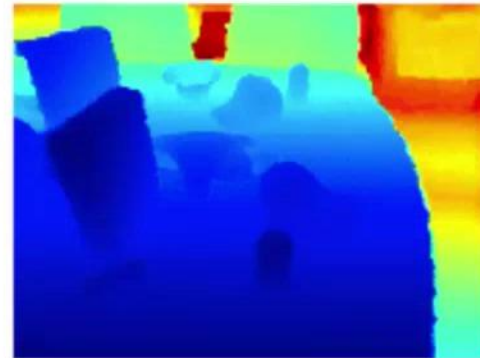
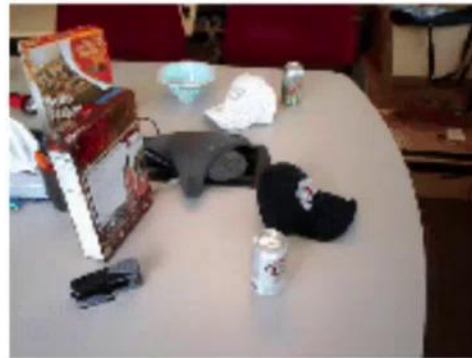
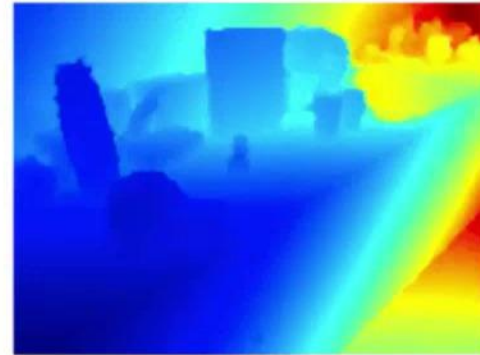
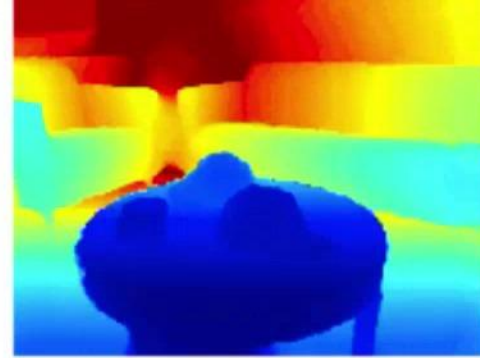
Cámaras de profundidad



90 mm x 25 mm x 25 mm

RGB-D

- RGB, cámara color común
- D: Depth
- Mide la distancia en cada pixel

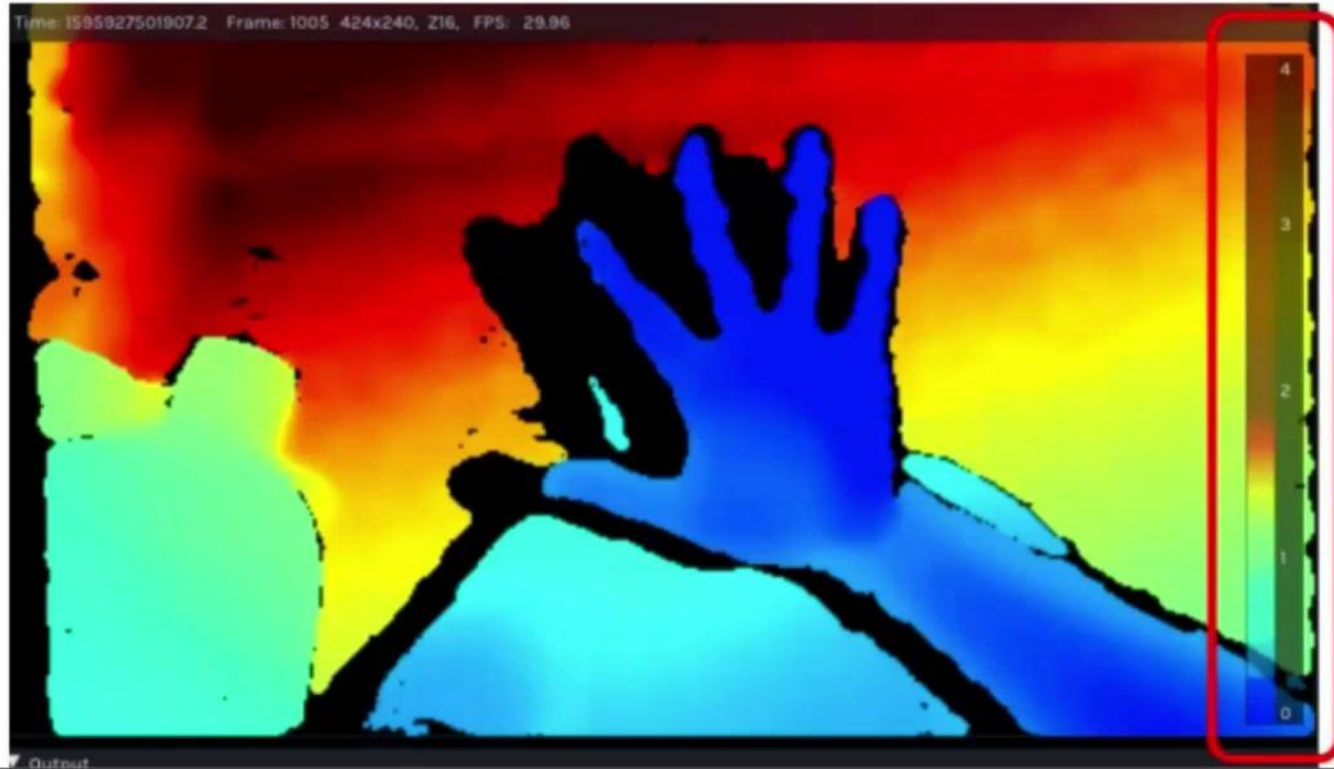


(a) RGB image

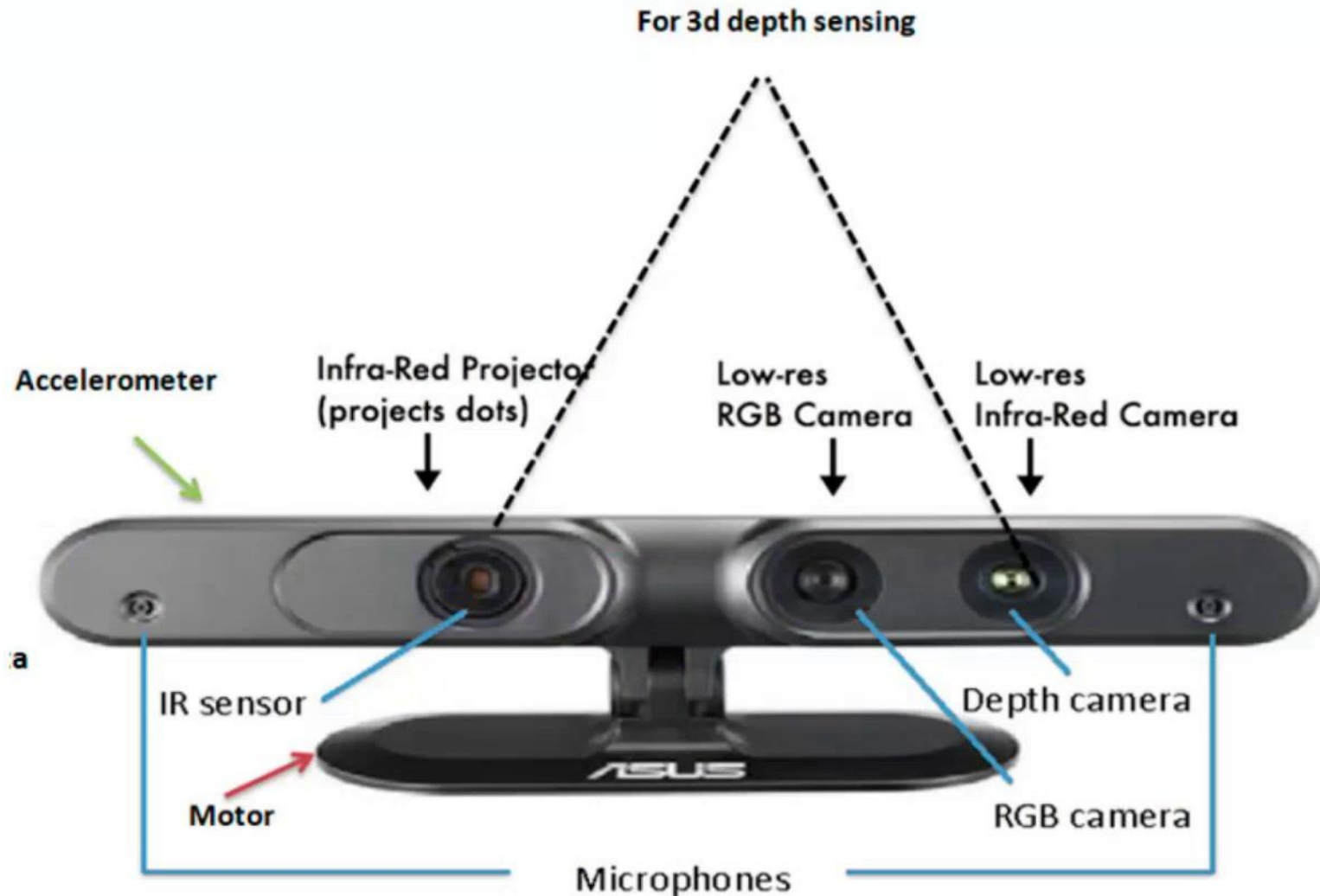
(b) Depth image

Mapa de profundidad

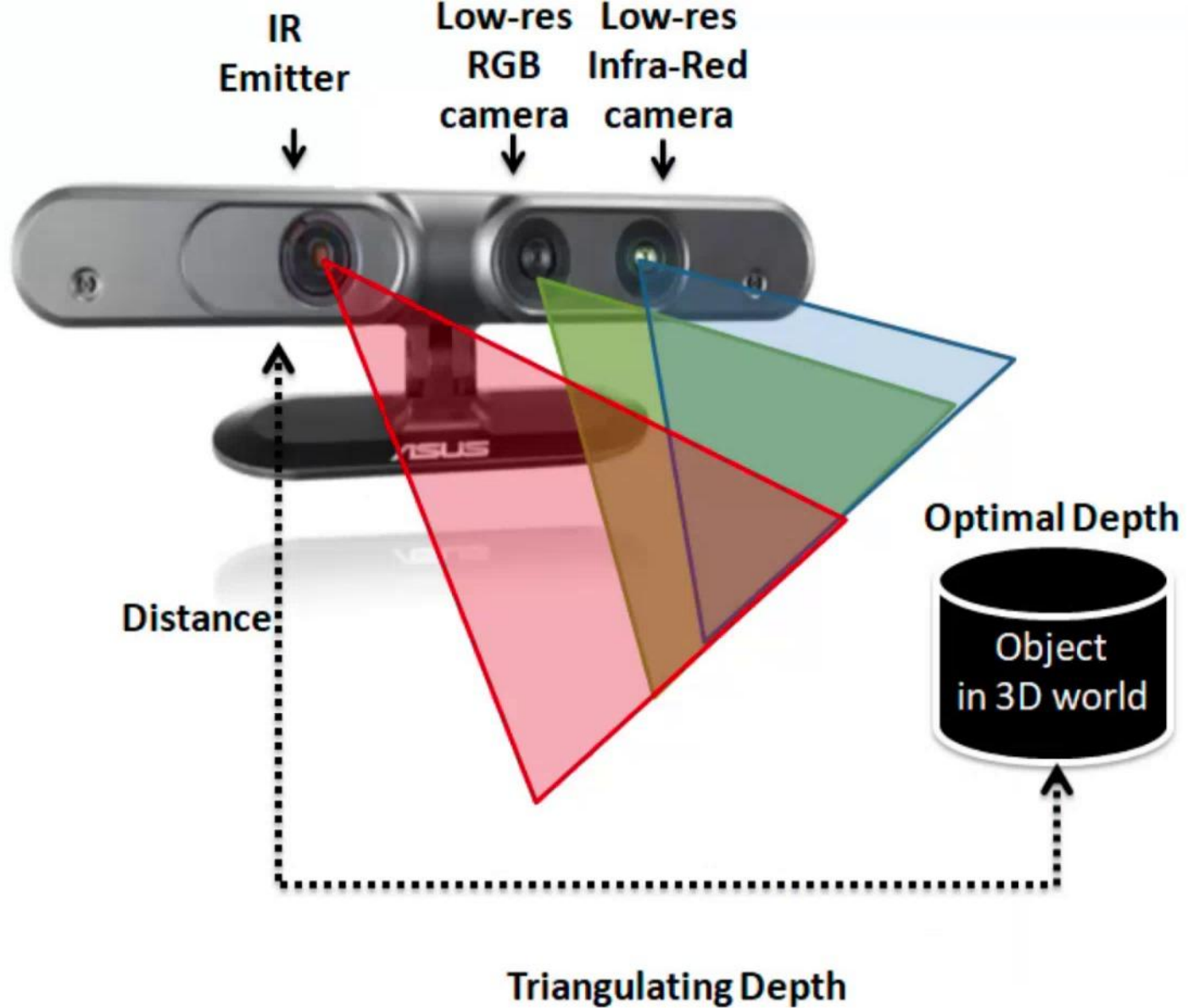
- Mide la distancia en cada pixel
- la imagen de profundidad tiene una resolución mucho menor que la imagen RGB
- algunos algoritmos interpolan la profundidad para cada píxel RGB



Sensores

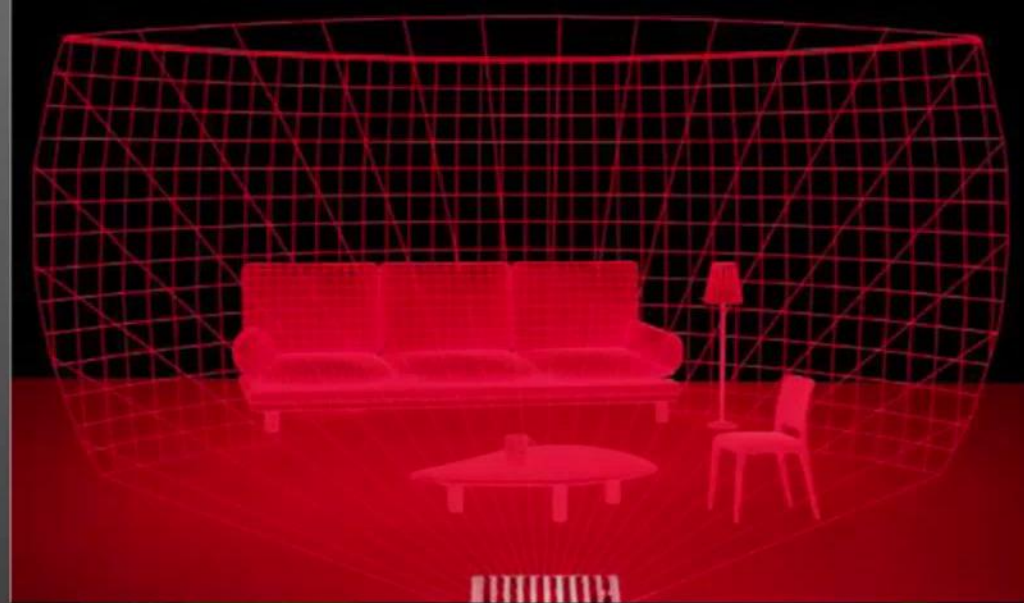


Paralaje



Luz estructurada

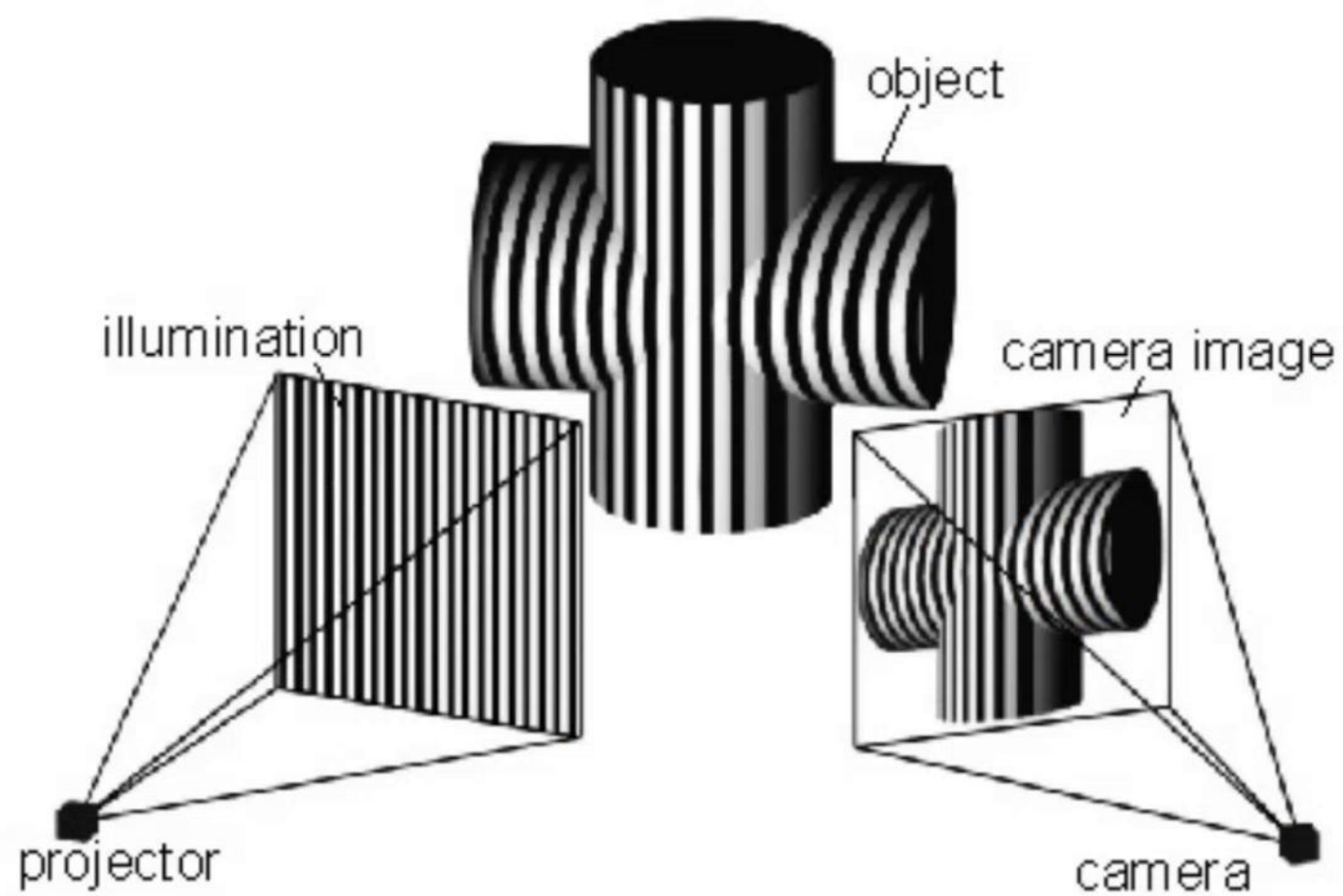
- Patrón geométrico proyectado sobre los objetos
- Para captar con una cámara el patrón proyectado
 - se necesita alto contraste
 - idealmente independiente del color de los objetos
- Luz fuerte en un ambiente oscuro
- Luz infrarroja



Patrón proyectado

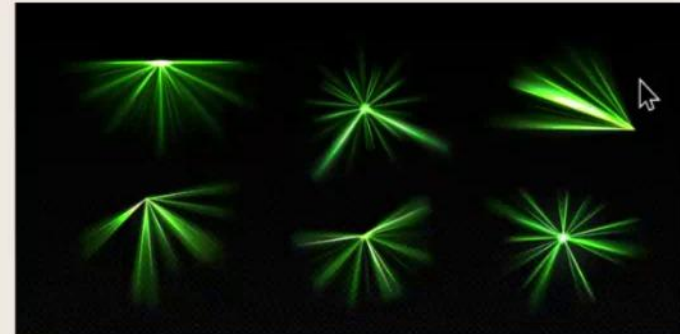
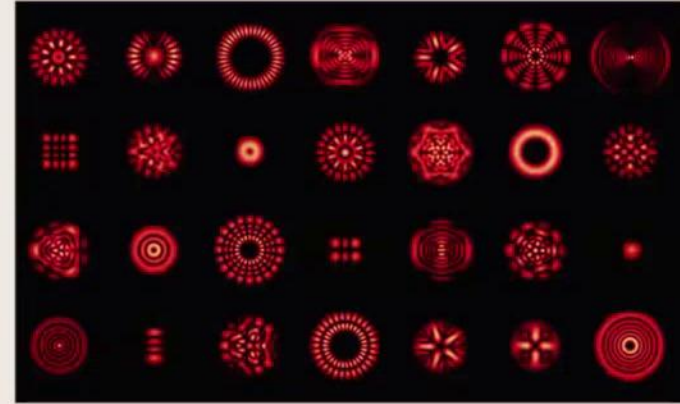
- Captado con una cámara, con paralaje
- En este ejemplo se relevan las líneas verticales
- Se proyectan líneas rectas, se relevan líneas curvas
- El apartamiento de la recta vertical permite medir la profundidad





Conclusiones

- Las cámaras RGB-D miden profundidad, pero no se debe confundir la resolución de la imagen con la de profundidad
- Miden por triangulación, de manera que se pierde definición cuando aumentan las distancias
- La potencia de la luz estructurada proyectada limita el alcance de la cámara



Referencias

- [Black-body radiation - Wikipedia](#)
- [How does Thermal Imaging Work? Night Vision & Through Walls](#)
- [This UV Camera Shows the Invisible](#)

SfM: Structure from Motion

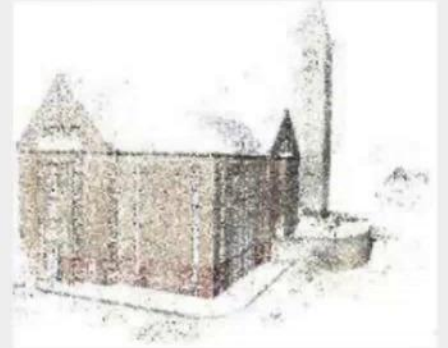
Definición Base

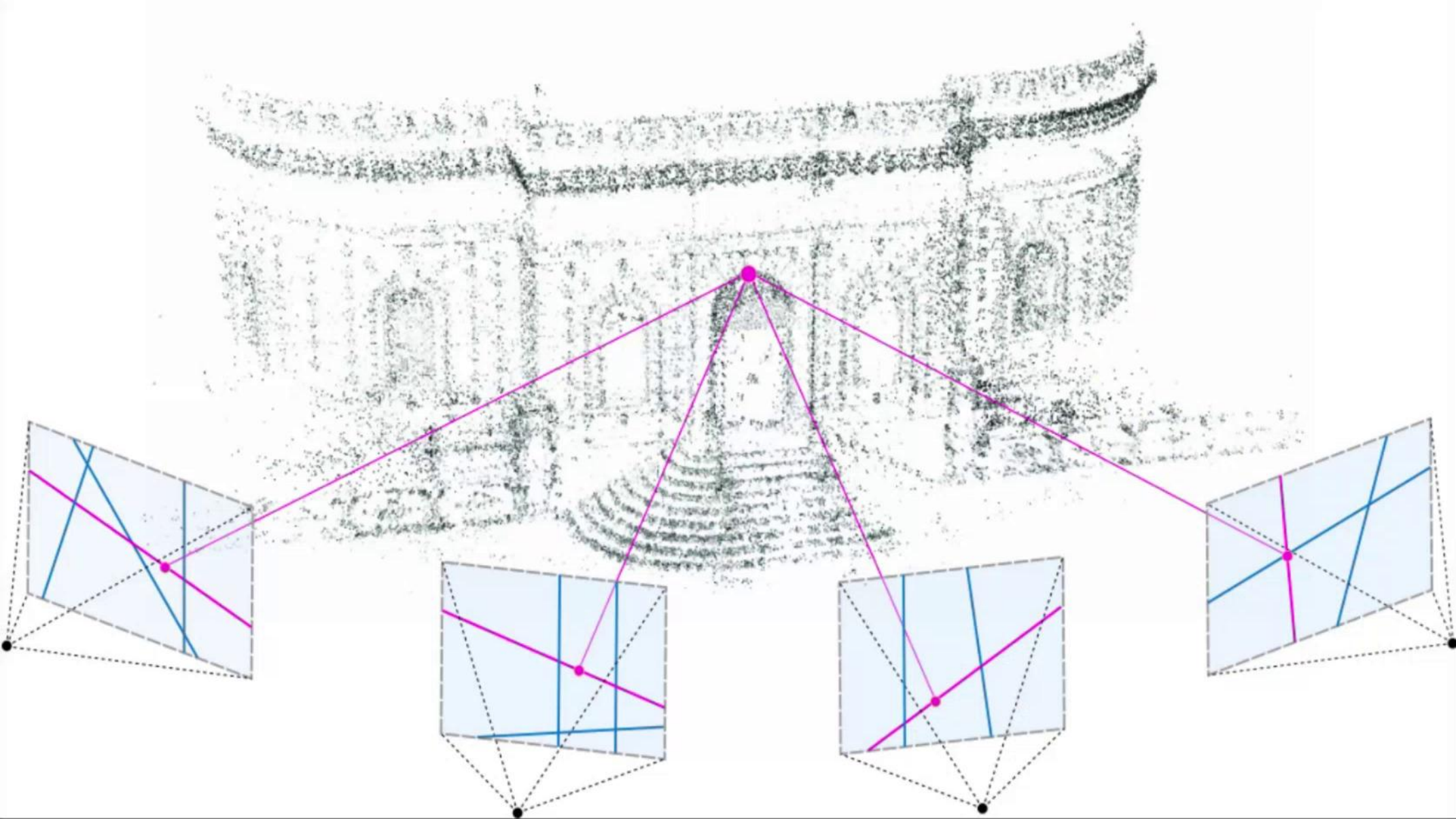
Obtención de la **estructura** a partir del **movimiento**.

Componentes

La estructura se compone de la **nube de puntos 3D** y las **poses de las cámaras**.

El **movimiento** es una secuencia de imágenes tomadas desde diferentes posiciones.





Reconstrucción 3D

Versatilidad de SfM

Structure from Motion no se limita al relevamiento de grandes entornos.

También es una herramienta potente para la reconstrucción detallada de objetos individuales.



Relevamiento de Entornos

Captura de Objetos

Marcadores fiduciarios

Ancla Visual

Los marcadores fiduciarios se fijan a la escena o a un objeto para servir como referencia.

Reconocimiento

Se reconocen fácilmente por algoritmos para determinar su pose relativa a la cámara.

Localización

Permiten saber dónde se encuentra la cámara o el objeto en el espacio.



Visual Place Recognition

Identificación

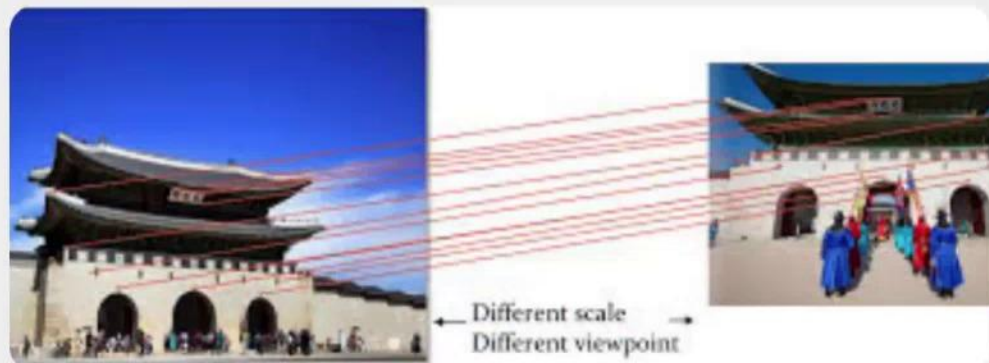
Reconoce un lugar a partir de una foto.

Geolocalización

Te dice dónde está ubicado exactamente.

Estimación de Pose

Algunas variantes calculan la pose precisa de la cámara.



Realidad aumentada

Escena en 3D

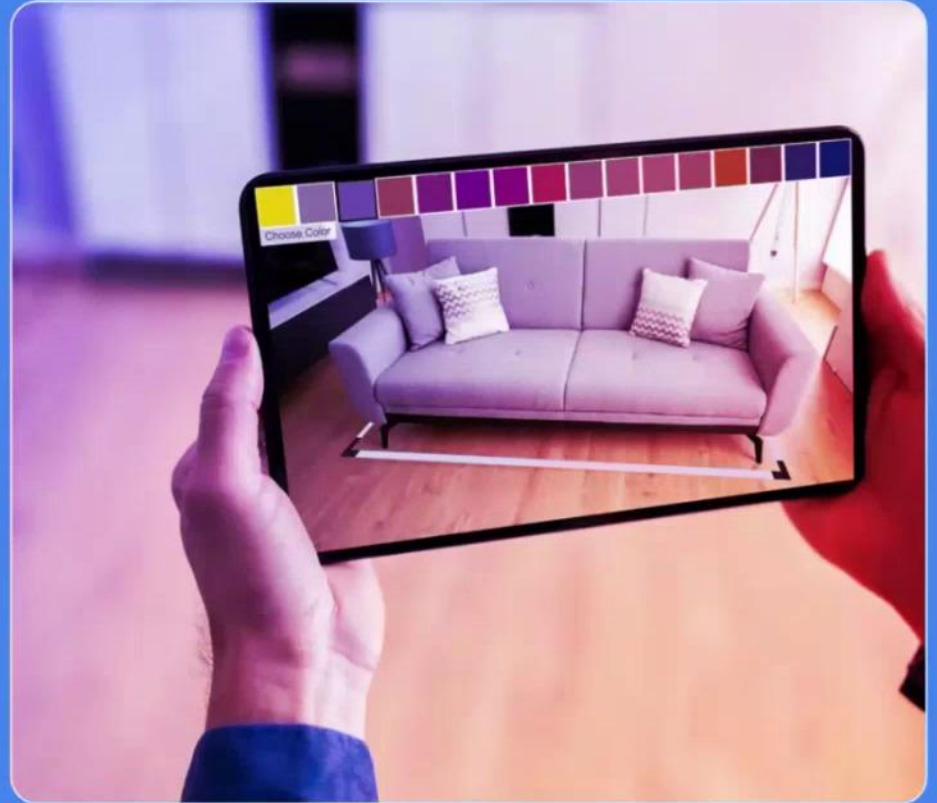
Comprende la pose de la escena en 3D para una integración espacial perfecta.

Modelado Preciso

Superpone un modelo 3D correctamente alineado con el entorno real.

Interacción en Tiempo Real

Permite una experiencia inmersiva mediante la sincronización visual avanzada.



Visual Odometry

Flujo Óptico

Analiza el flujo óptico para determinar el desplazamiento relativo del sensor.

Cinemática del Video

Calcula traslación y rotación 3D entre dos cuadros consecutivos de un video.



Visual Inertial Odometry

Fusión Sensorial (VIO)

Combina cámaras con sensores IMU para estimar movimiento con escala real precisa.

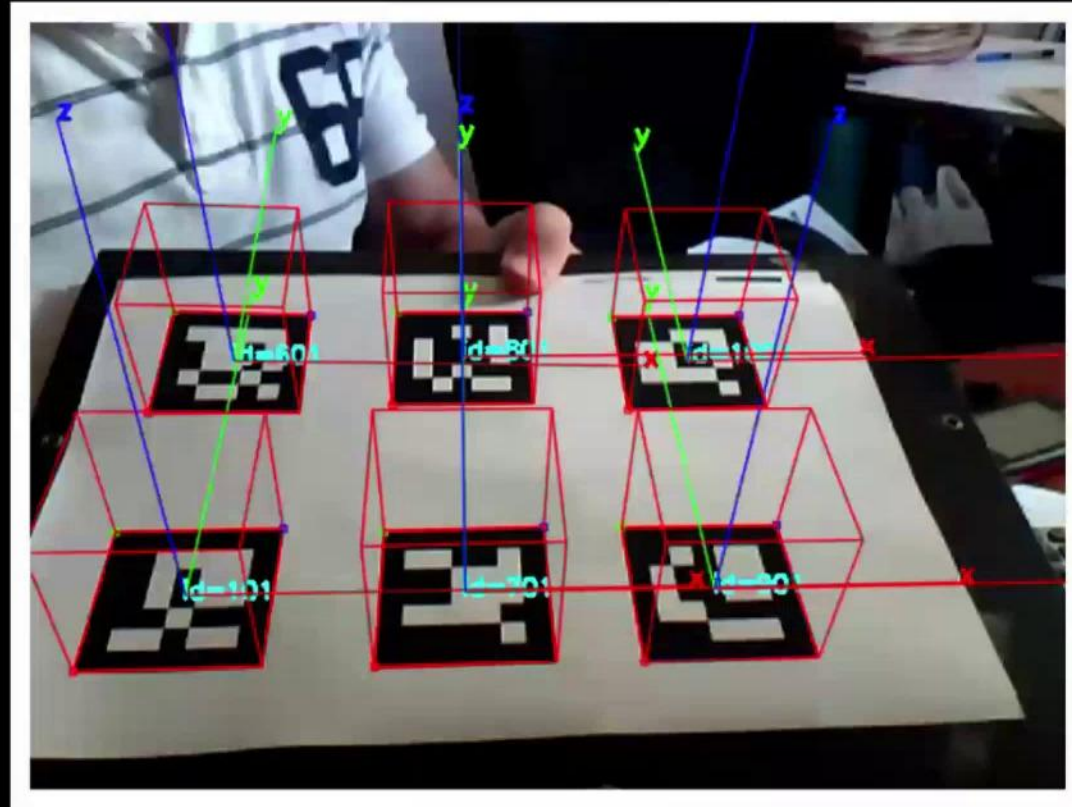
Integración Nativa

ArCore ofrece VIO en Android listo para usar, eliminando la necesidad de claves API complejas.

Futuro en la Web

WebXR democratiza estas capacidades avanzadas directamente desde el navegador.





Perspective from n Points

Alejandro Silvestri

¿Para qué sirve obtener la pose 3D de un objeto en una imagen?

- Realidad aumentada
 - conociendo la pose se puede generar un modelo virtual con la misma perspectiva
- Posicionamiento externo
 - conocida la pose de la cámara en el mundo, se puede obtener la pose del objeto en el mundo
- Autoposicionamiento
 - conocida la pose en el mundo de un objeto de referencia, un robot móvil con cámara puede deducir su propia pose en el mundo





Cómo obtener la posición 3D de un objeto en una escena, con una cámara

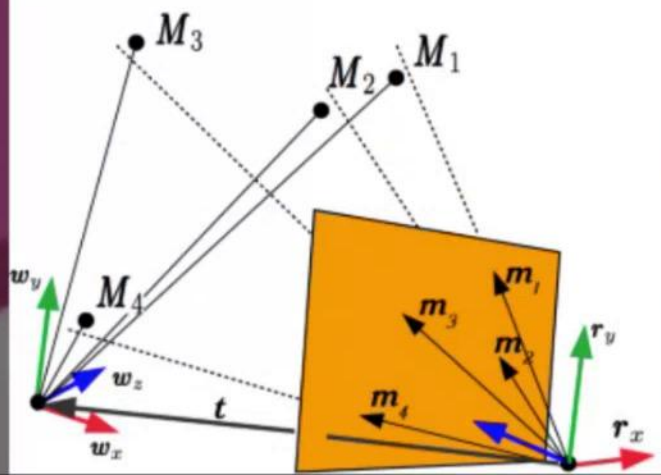


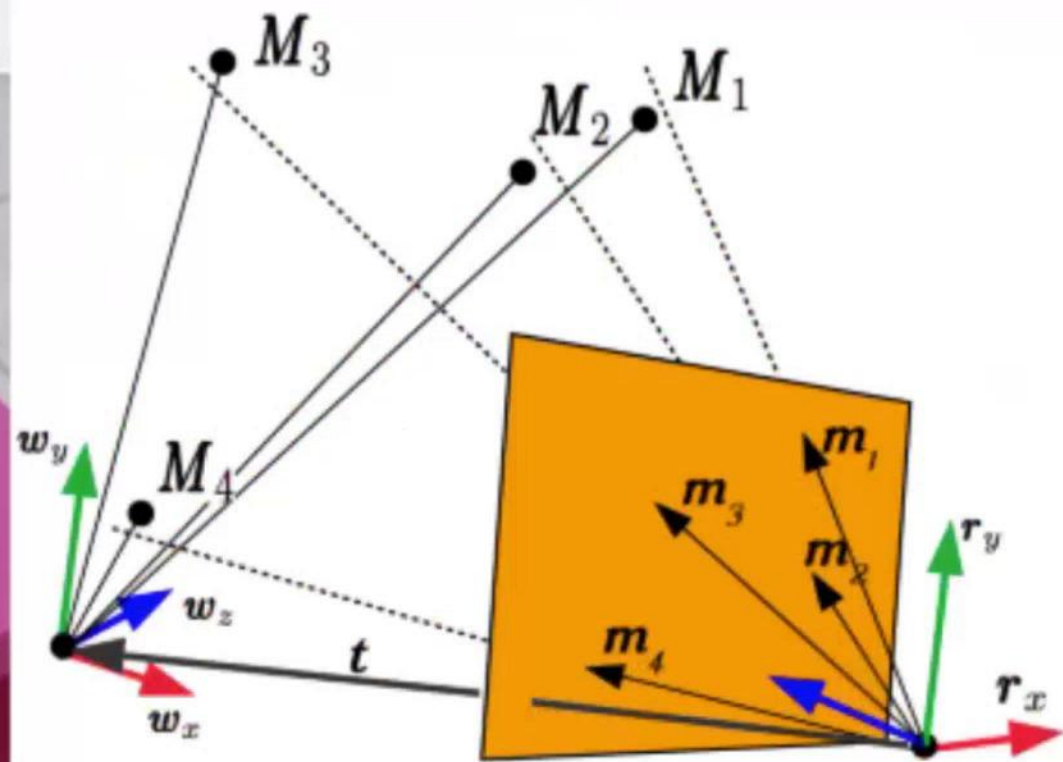
- se necesita un **sistema de referencia del mundo** para poder expresar la posición 3D
- de una imagen sólo se obtienen **coordenadas 2D** en píxeles
- una imagen **no tiene información de escala**, y no se conoce la distancia de los objetos a la cámara; se requiere información adicional a la presente en una imagen
- la **posición de un punto** se expresa con un vector 3D, la **pose de un cuerpo rígido** expresa con una matriz, requieren métodos diferentes
- las distorsiones de la lente cambian la dirección del rayo de luz, y altera la medición

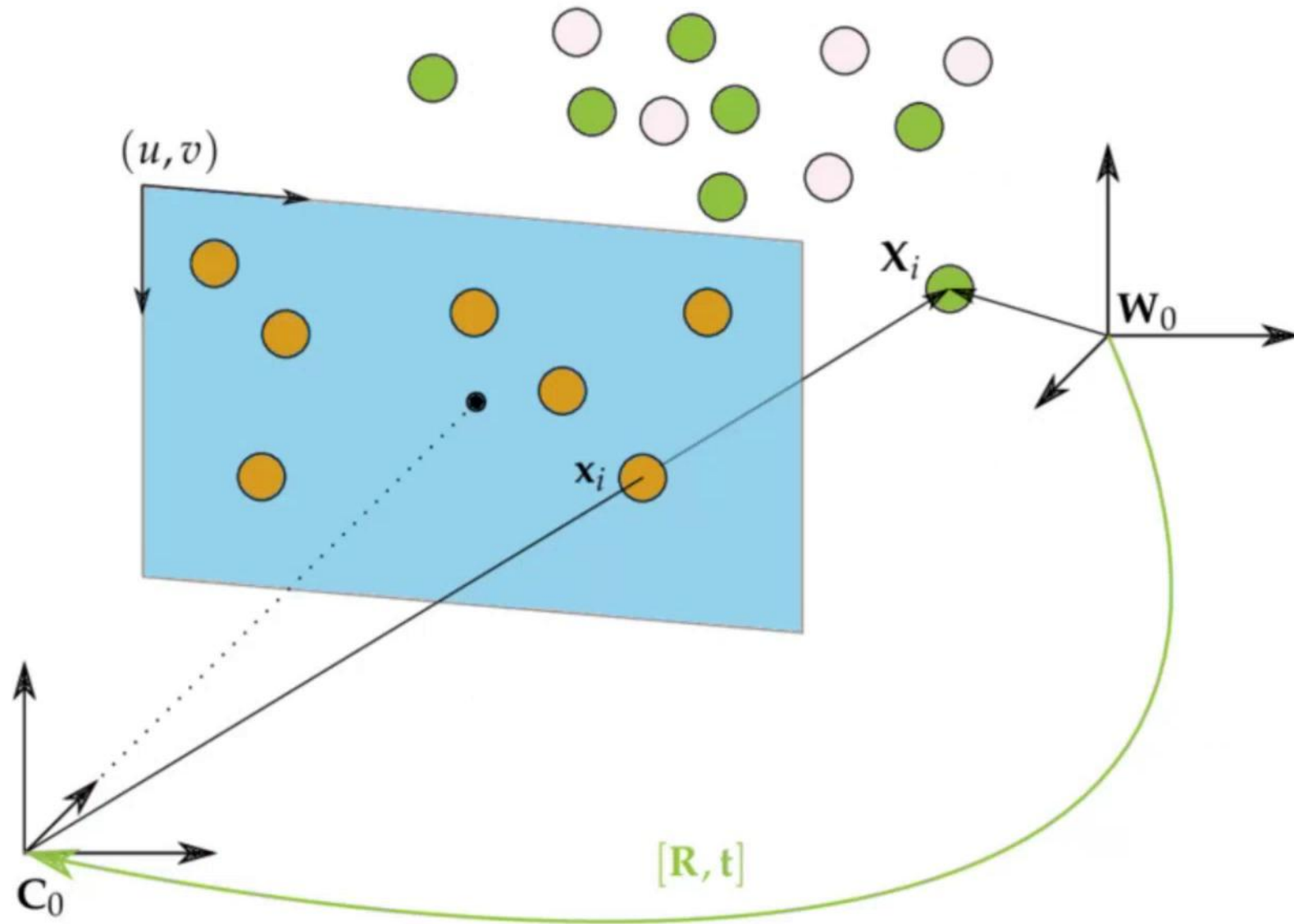
PnP

Perspectiva a partir de n Puntos

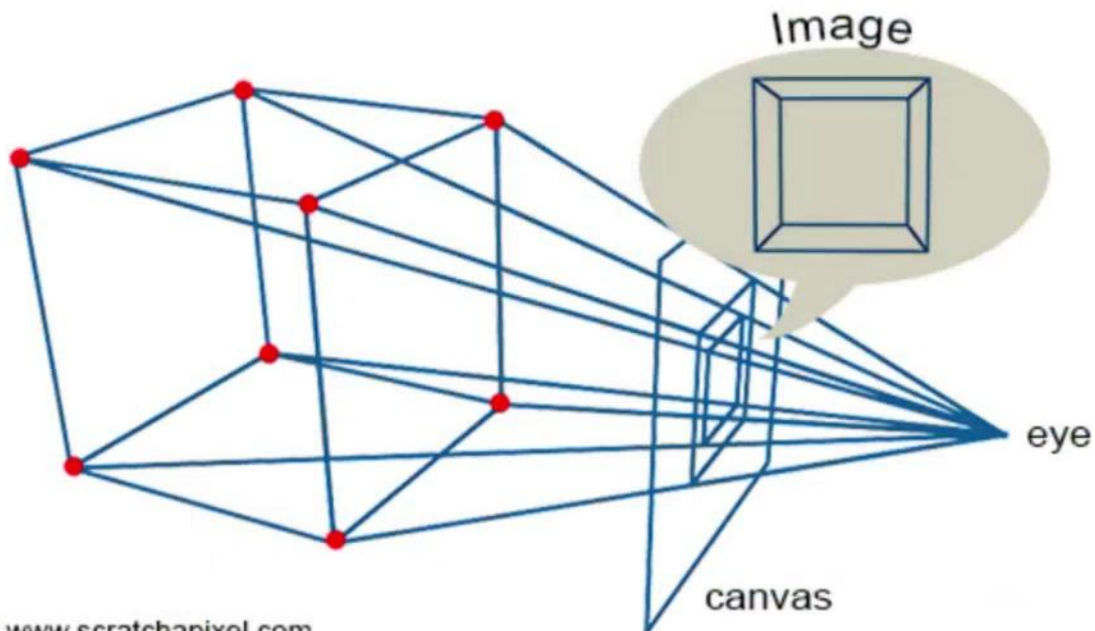
- Datos
 - coordenadas 3D de puntos de un modelo de cuerpo rígido
 - coordenadas 2D de los mismos puntos vistos en una imagen
- PnP obtiene
 - la pose de un cuerpo rígido a partir de una sola imagen
 - rotación y traslación del cuerpo rígido respecto de la cámara





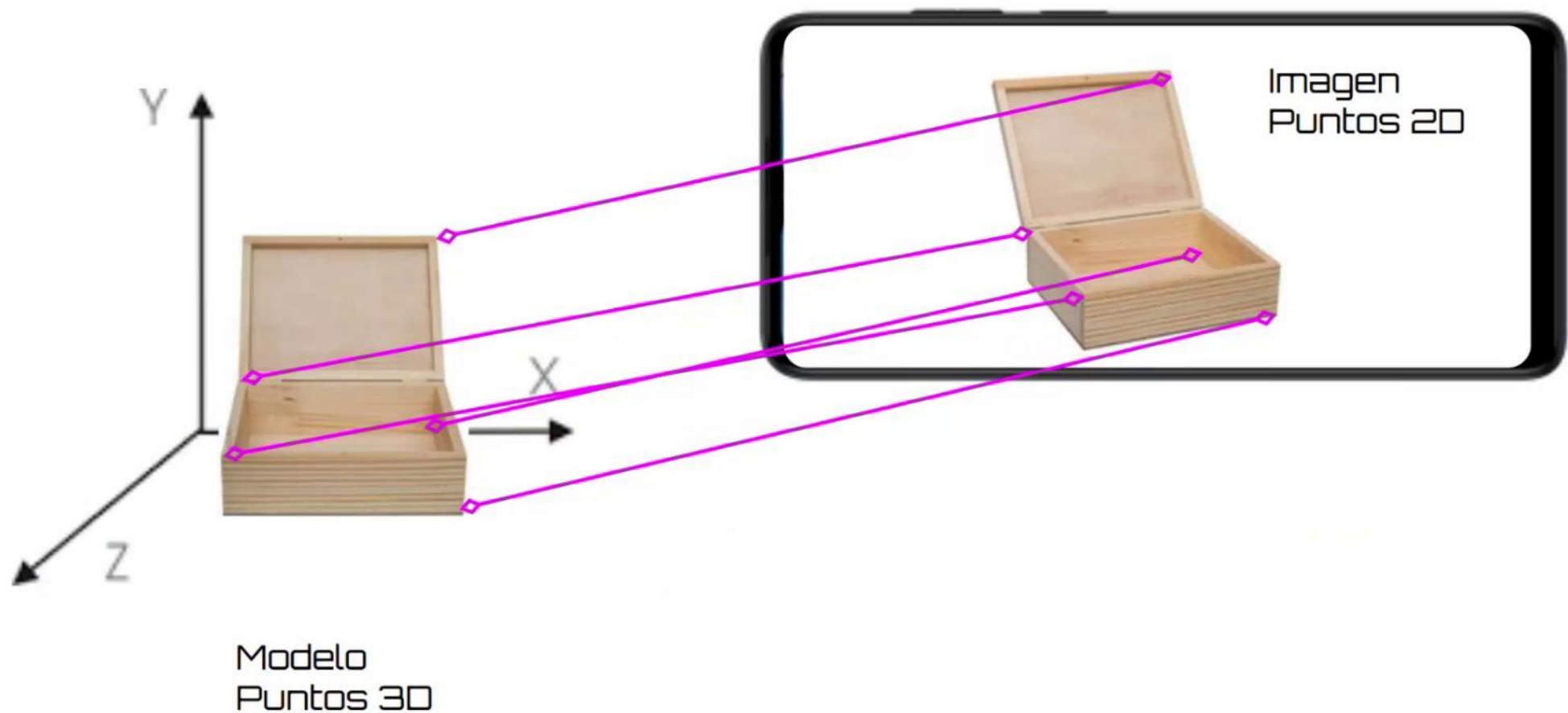


Datos necesarios



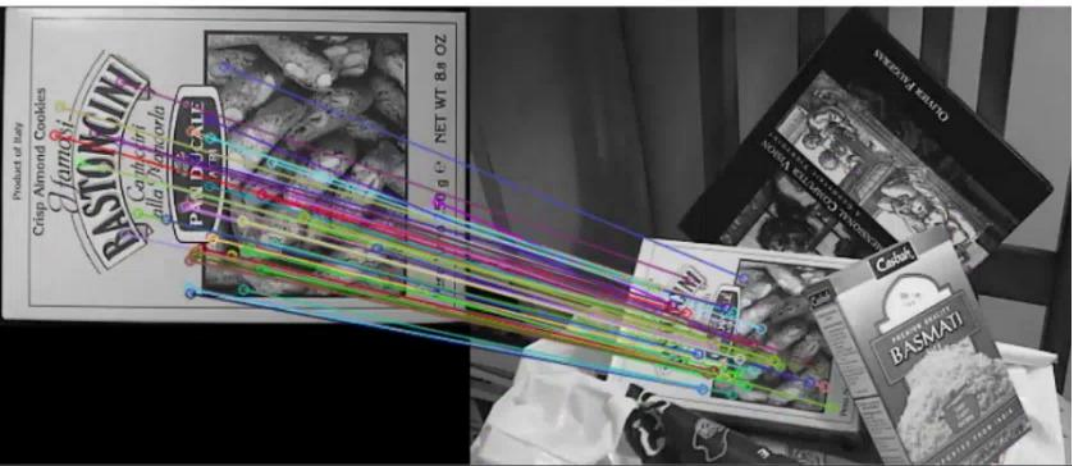
- Correspondencias
 - puntos 2D detectados en la imagen
 - puntos 3D correspondientes del modelo (del cuerpo rígido)
 - ¿cuántas correspondencias?
- Calibración de cámara
 - matriz intrínseca
 - parámetros de distorsión

Correspondencias 3D-2D



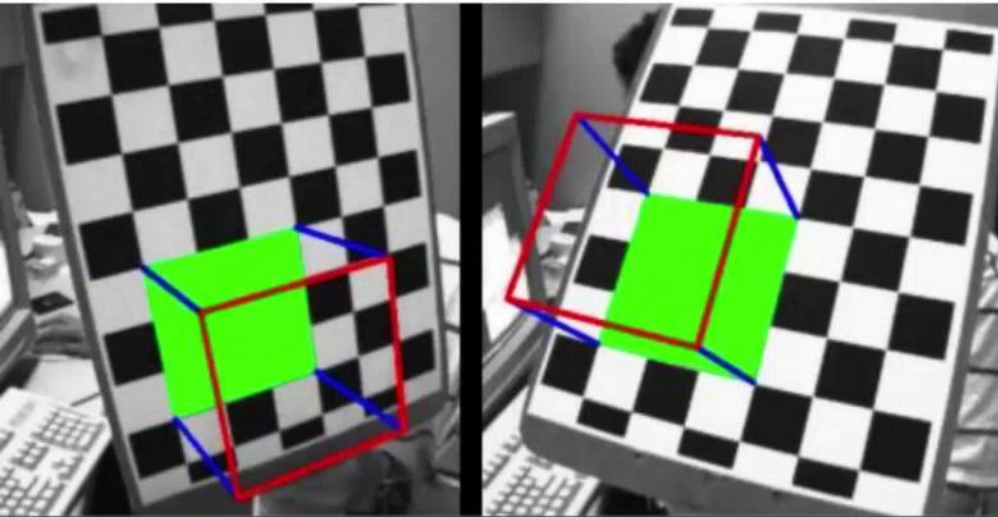
¿Cuántas correspondencias se necesitan?

- Mínimo 3
 - sistema completamente determinado
 - 4 soluciones posibles
- Usualmente 4
 - solución única
 - excepcionalmente 2 soluciones en singularidades
- Más de 4
 - métodos de minimización de errores
 - métodos robustos



Resultados

- Traslación y rotación
 - vector traslación 3D
 - **en la escala del modelo**
 - orientación 3D, en notación Rodriguez
- Ambigüedades
 - pueden haber hasta 4 resultados



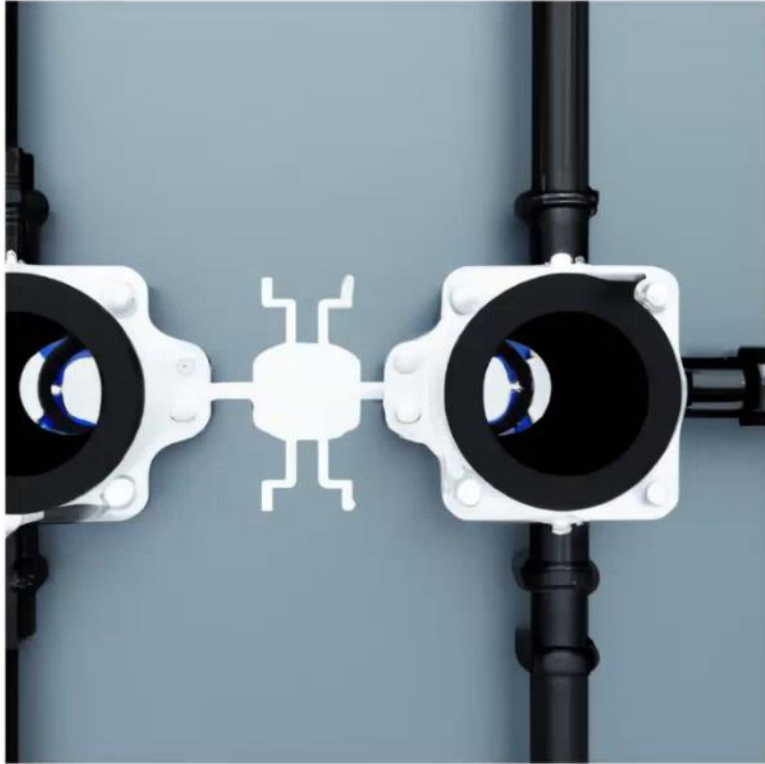
Varias soluciones

pero sólo una es la correcta



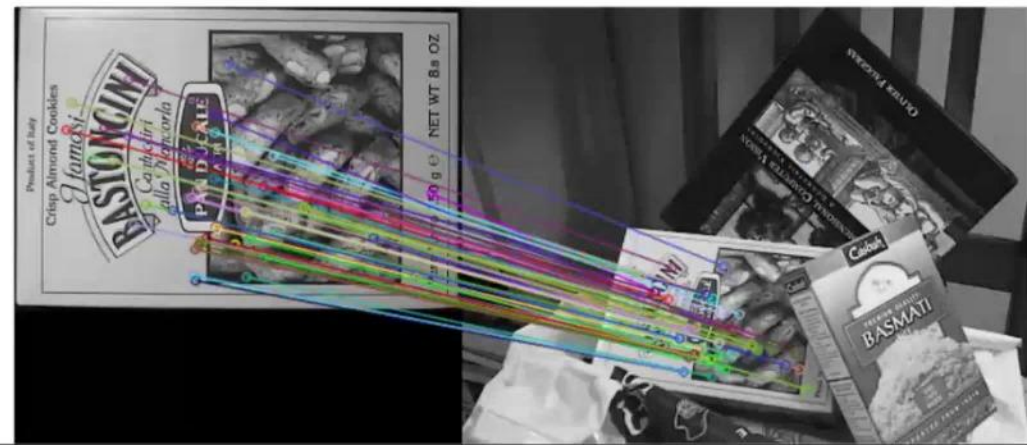
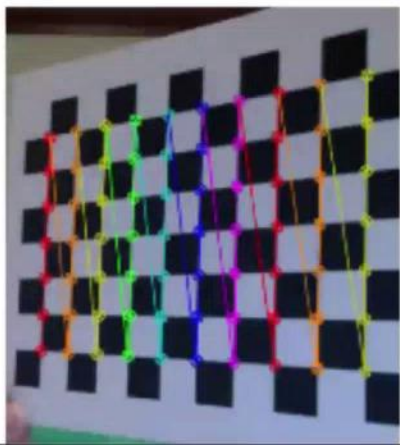
- Caso degenerado
 - los puntos del modelo están alienados
 - no sirven para determinar pose
- Singularidades
 - 4 soluciones ambiguas para el caso mínimo de 3 correspondencias
 - 2 soluciones para IPPE e IPPE_SQUARE en ciertos casos particulares
- Elección de la solución correcta
 - descartar las poses que sitúan los puntos 3D detrás de la cámara
 - usar información adicional

Pipeline



- Setup
 - calibración de cámara
 - matriz intrínseca
 - coeficientes de distorsión
 - modelo 3D
 - coordenadas de puntos para machear
- Loop
 - detección de puntos 2D en la imagen
 - correspondencias 2D-3D
 - **solvePnP**
 - se obtienen traslación y rotación
 - eventualmente, descarte de soluciones ambiguas
 - anotación

Soluciones



- Cada solución determina de manera particular:
 - el **preprocesamiento**
 - la detección de puntos 2D
 - el macheo 2D-3D
 - el **algoritmo PnP** específico
- Algunas soluciones
 - pose de **marcadores** QR y ARUCO
 - pose de **patrón** de calibración
 - pose **sin marcadores**
 - pose de cuerpo rígido **balizado**

Biblioteca PnP



- **solveP3P**
 - 3 correspondencias
 - 4 soluciones
- **solvePnP**
 - 4 o más correspondencias
 - varios algoritmos
 - 1 solución (si hay varias elige una)
- **solvePnPGeneric**
 - similar al anterior, todas las soluciones encontradas
- **solvePnP Ransac**
 - único método robusto

Colección de algoritmos de PnP

- P3P, AP3P
 - 4 correspondencias
 - especializados en mínima cantidad de correspondencias
- IPPE
 - especializado en **modelo plano** (puntos coplanares)
- IPPE_SQUARE
 - IPPE **especializado en cuadrado**
 - 4 correspondencias en posiciones predeterminadas

Cada algoritmo tiene su aplicación

- EPNP
 - es el algoritmo más **genérico**
 - 6 o más correspondencias
 - presenta **robustez** frente a outliers
- SQPNP
 - el más **moderno** (2020)
 - 3 o más correspondencias
- ITERATIVE
 - método **antiguo** que emplea un optimizador LM para minimizar el error de reproyección
 - se beneficia de un valor inicial para el optimizador
 - usa homografía para modelos planos y DLT para modelos 3D
 - los algoritmos más nuevos son mejores en velocidad y precisión
 - es el algoritmo por defecto, no es el mejor en nada pero que abarca todos los casos

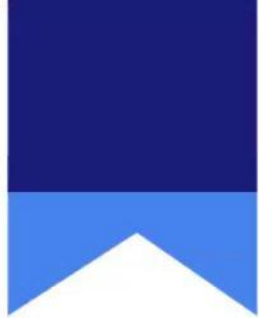
Código ejemplo

```
retval, rvec, tvec = cv.solvePnP(objectPoints, imagePoints,  
cameraMatrix, distCoeffs, flags=cv.SOLVEPNP_IPPE_SQUARE)
```

- argumentos
 - objectPoints: puntos 3D del modelo
 - imagePoints: puntos 2D en píxeles
 - cameraMatrix: matriz de calibración de cámara
 - distCoeffs: coeficientes de distorsión
 - flags: algoritmo PNP
- resultados
 - rvec: rotación en notación Rodrigues
 - tvec: traslación en la escala del modelo

Anotación 3D

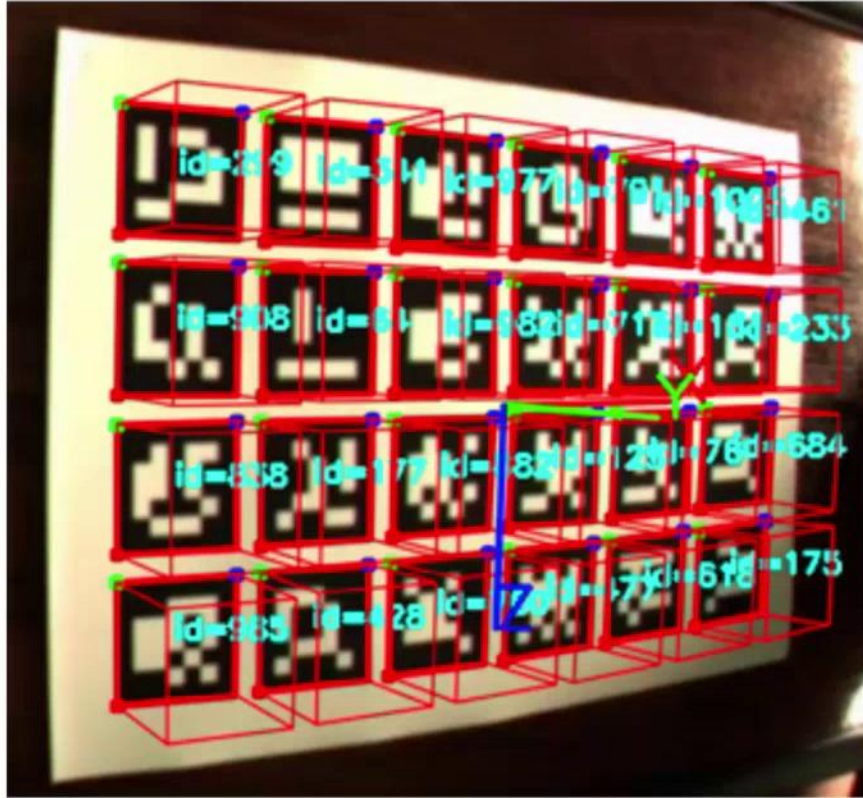




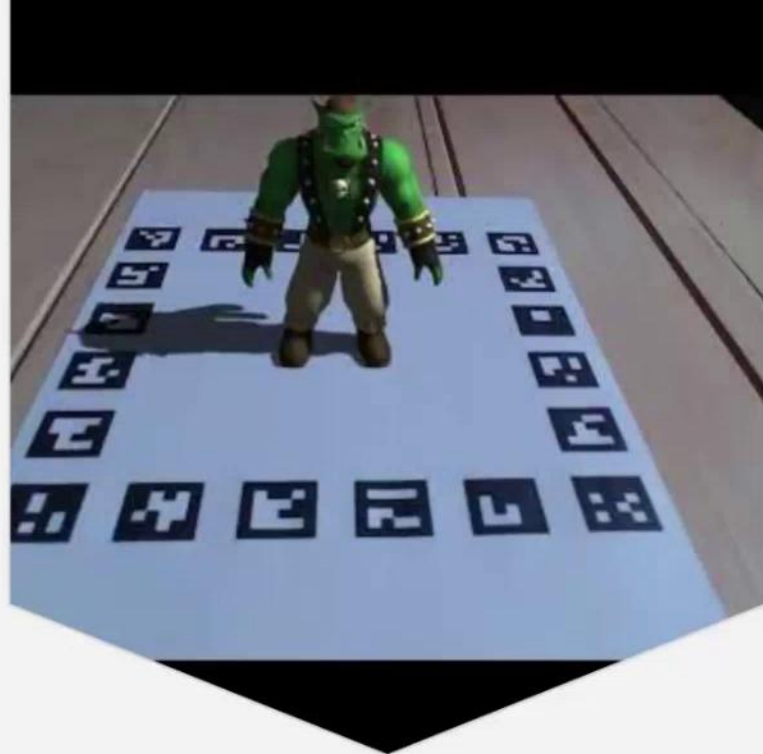
Referencias

- [Perspective-n-Point \(PnP\) pose computation](#)
- [solvePnP\(\)](#)
- [Example of pose estimation of calibration pattern](#)
 - código ejemplo que anota ejes de coordenadas y cubos

Anotación de un cubo



- Setup
 - coordenadas 3D de los **8 vértices** del cubo en el sistema de referencia del modelo 3D
- Loop
 - pasar los 8 puntos al **sistema de referencia de la cámara**
 - usando R, t obtenido por PnP
 - **proyectar** estos 8 puntos con la matriz intrínseca de la cámara
 - se obtienen 8 puntos 2D en coordenadas homogéneas, en píxeles
 - *opcionalmente, **distorsionar** los puntos para alinear con la distorsión de la lente*
 - **anotar** los 12 segmentos de recta de las **aristas** del cubo



Pose 3D de un marcador

Alejandro Silvestri

Pipeline de procesamiento

1. **detección de puntos 2D en la imagen**
 - a. los métodos varían según el caso
2. **macheo de estos puntos con una referencia**
 - a. los puntos no macheados se descartan
3. **PnP**
 - a. con el algoritmo más adecuado para cada solución



QR

- QR: Quick Response
- el detector localiza los 3 patrones principales de posición y varios otros intercalados en el código
- luego computa la localización de los **4 vértices del cuadrado virtual** contenedor

```
retval, points = cv.QRCodeDetector.detect(img)
```

$$P0=(-L/2; -L/2; 0)$$

$$P1=(L/2; -L/2; 0)$$

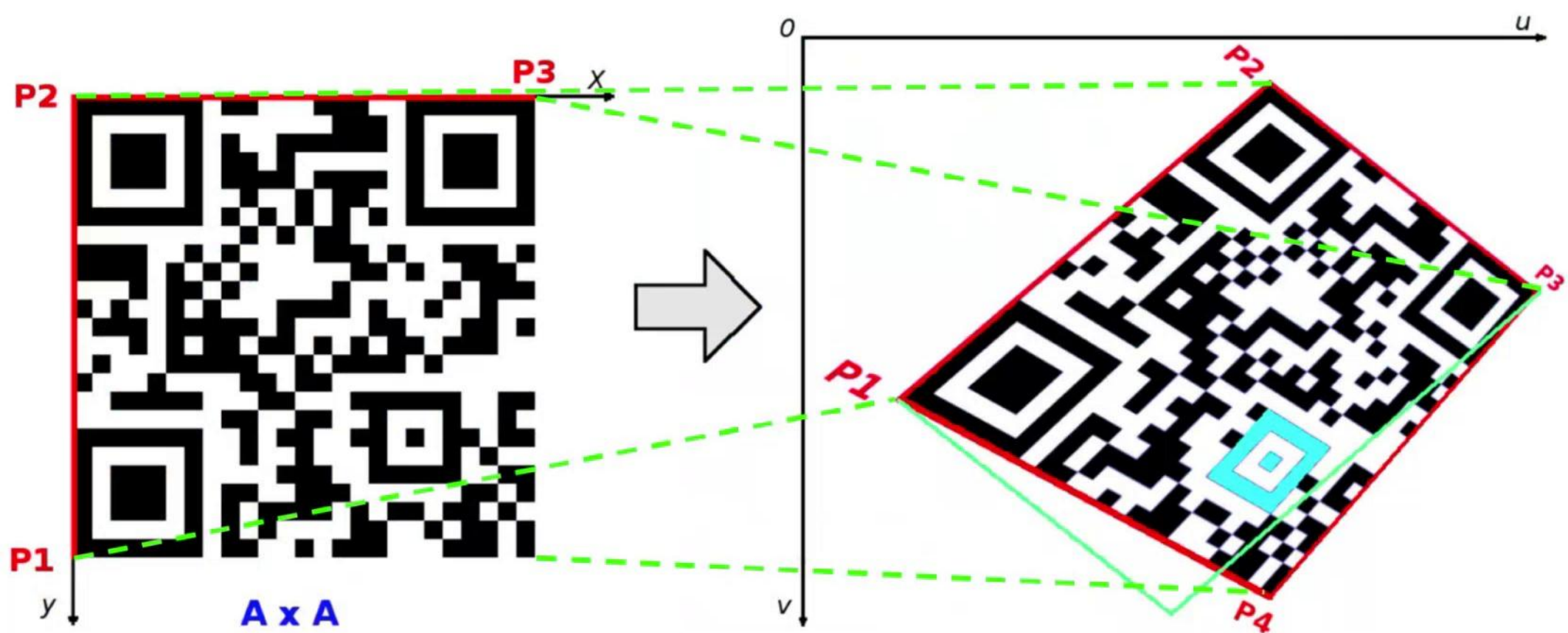
QR como sistema de coordenadas

- L es el lado del cuadrado en la escala del mundo real
- PnP expresará la traslación t en esa misma escala
- Los puntos P0 a P3 se numeran según el orden establecido por el algoritmo PnP IPPE Square



$$P3=(-L/2; L/2; 0)$$

$$P2=(L/2; L/2; 0)$$



Correspondencias

- izquierda, puntos en las coordenadas del patrón
- derecha, puntos en las coordenadas de la imagen

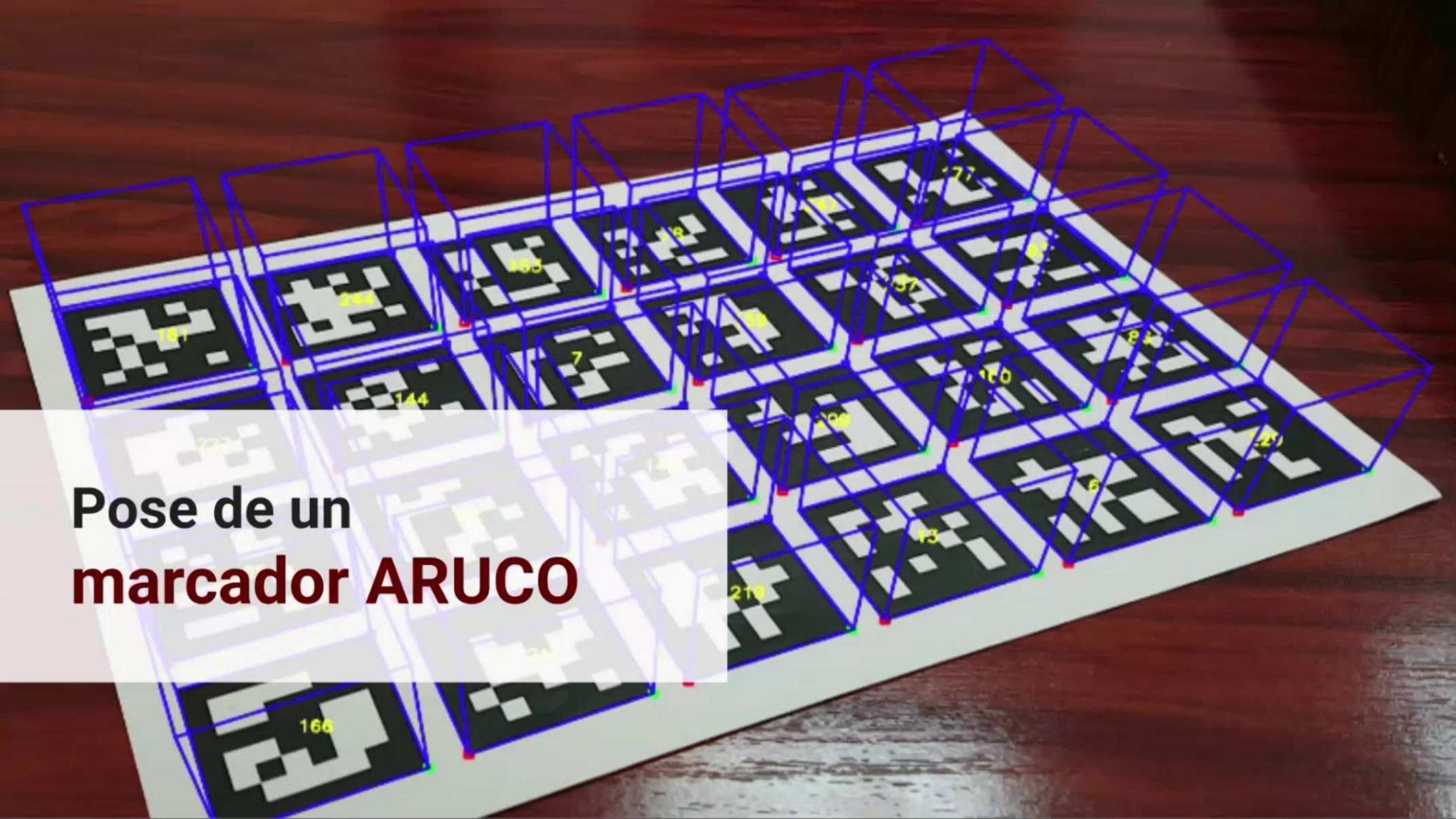
Código ejemplo



- La biblioteca tiene un método propio para detectar los puntos de un código QR
 - `cv.QRCodeDetector.detect(img)`
- El algoritmo PnP adecuado es **IPPE_SQUARE**
 - `cv.SOLVEPNP_IPPE_SQUARE`
 - busca exclusivamente la pose de un cuadrado

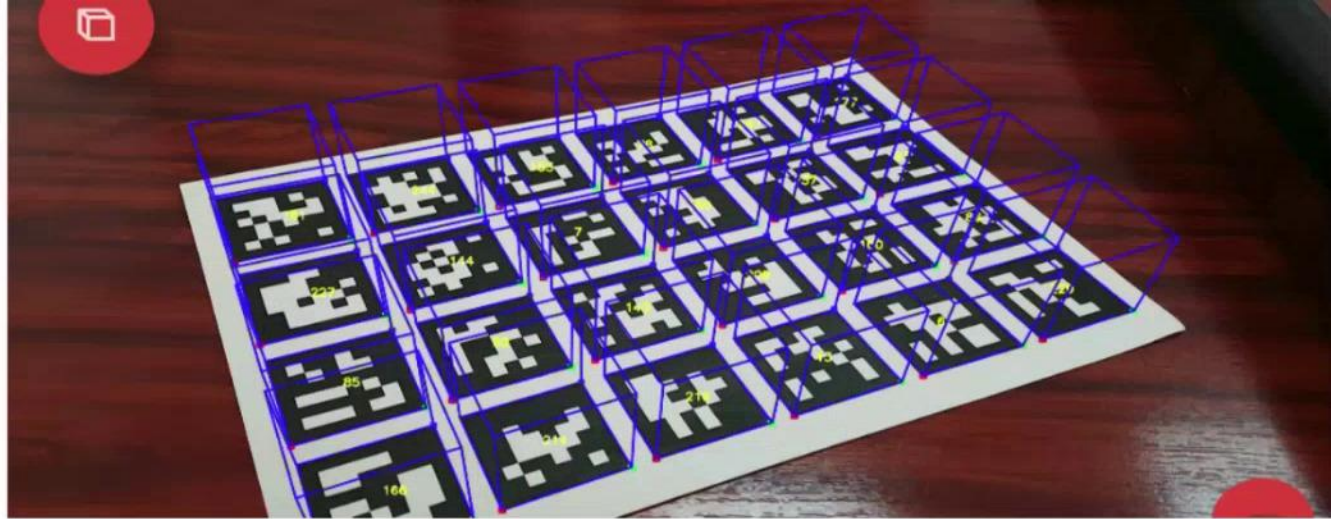
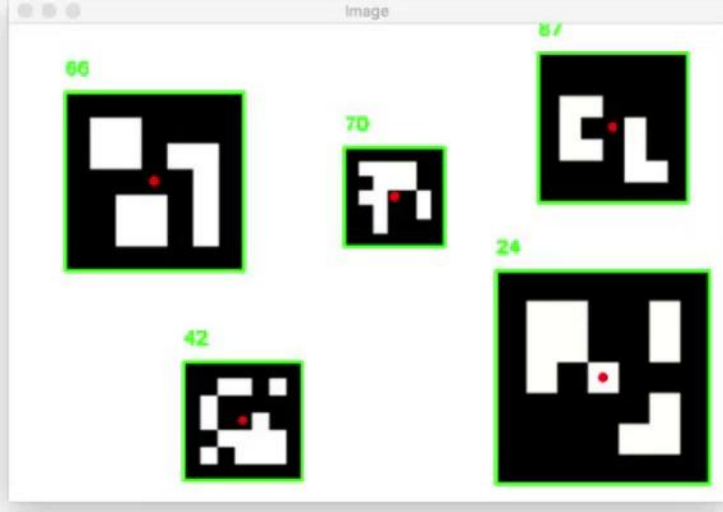
```
retval, imagePoints = cv.QRCodeDetector.detect(img)
```

```
retval, rvec, tvec = cv.solvePnP(objectPoints, imagePoints,  
cameraMatrix, distCoeffs, flags=cv.SOLVEPNP_IPPE_SQUARE)
```



The image shows a top-down view of a wooden surface with a white grid of ARUCO markers. Each marker is a square with a black and white pixelated pattern. A blue wireframe cube is overlaid on each marker, representing its 3D pose. The cubes are slightly tilted, indicating they are not perfectly aligned with the grid. Yellow numbers are printed on the top face of each cube, representing a unique identifier for each marker. The numbers are: 167, 233, 155, 9, 77, 8, 144, 7, 157, 23, 150, 8, 225, 12, 210, 13, 6, 21, 166, and 210. A semi-transparent white box with black text is overlaid on the left side of the image.

Pose de un
marcador ARUCO

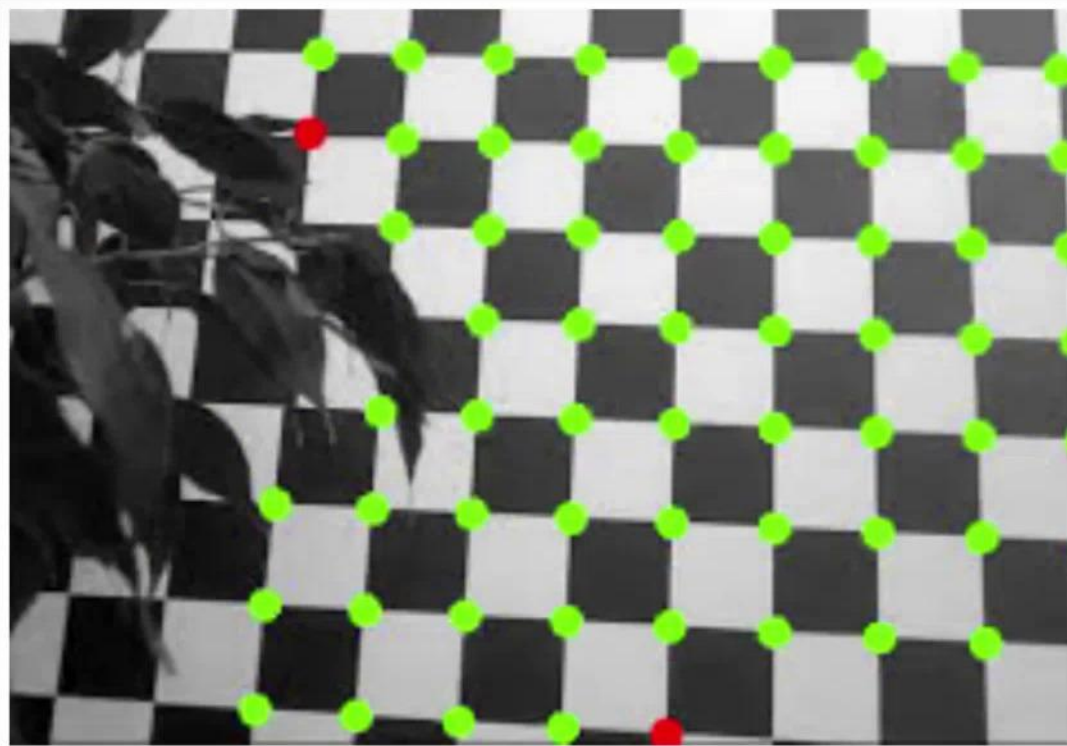


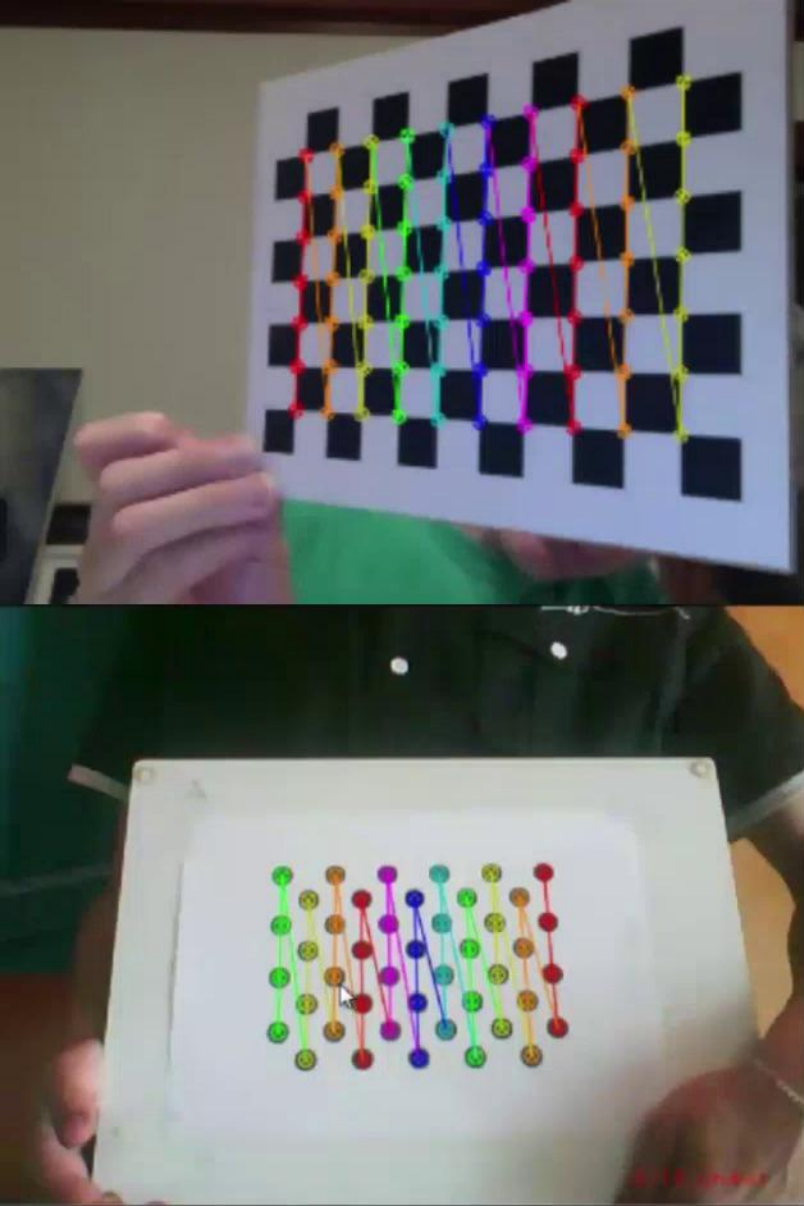
ARUCO

- marcadores ARUCO
 - codifican un símbolo que sirve de identificador
- ARUCO
 - significa Augmented Reality Universidad de Córdoba (España)
- el detector devuelve los **4 vértices del marcador** cuadrado
- al igual que con QR, los vértices se numeran según **PnP IPPE Square**

```
corners, ids, rejectedImgPoints = cv.aruco.ArucoDetector.detectMarkers(image)
```

Pose de un patrón de calibración

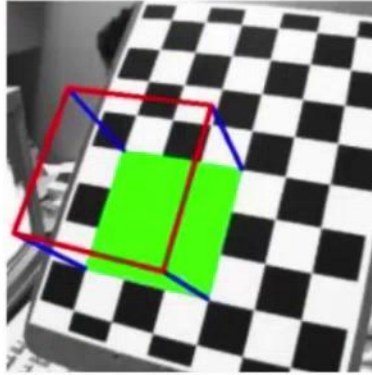
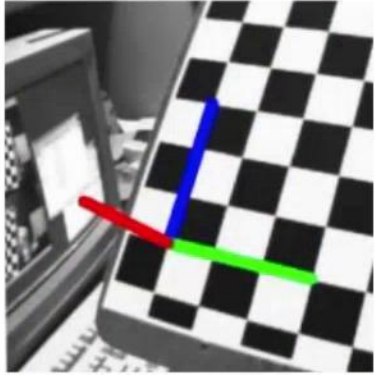




Detección de puntos en patrón de calibración

- los detectores de patrones de calibración devuelven las coordenadas de cada punto de interés del patrón
- en la imagen superior, en patrón **ajedrez** se detectan vértices internos, donde confluyen dos cuadrados negros y dos blancos
- en la imagen inferior, el patrón de **círculos** detecta las circunferencias y determina sus centros

Código ejemplo

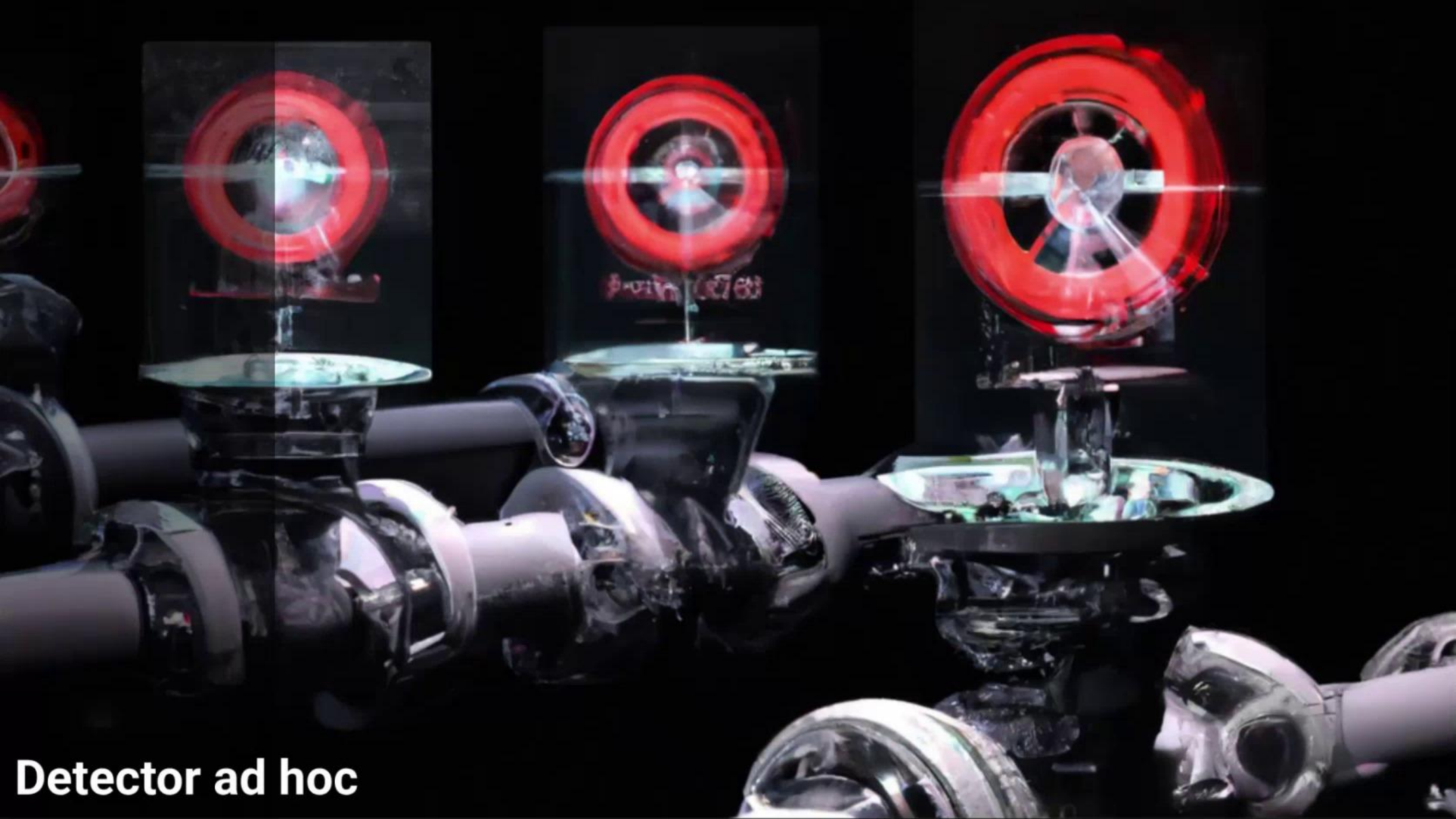


- La biblioteca de calibración tiene métodos propios para detectar puntos y refinarlos
 - `cv.findChessboardCorners()`
 - `cv.cornerSubPix()`
- El algoritmo PnP adecuado es EPNP
 - `cv.SOLVEPNP_EPNP`
 - alternativamente, es común usar el algoritmo por defecto
- Este tutorial demuestra cómo obtener la pose y anotarla con ejes cartesianos y cubos
 - [Pose Estimation](#)

```
retval, corners = cv.findChessboardCorners(grayImage, patternSize)

corners2 = cv.cornerSubPix(grayImage, corners, (11,11), (-1,-1), criteria)

retval, rvec, tvec = cv.solvePnP(objectPoints, corners, cameraMatrix,
distCoeffs, flags=cv.SOLVEPNP_EPNP)
```



Detector ad hoc



Detección de un punto 2D

- los puntos a detectar pertenecen a la escena, no a la imagen
 - si la escena se desplaza respecto de la cámara, el punto se desplaza con ella
- el punto no es necesariamente visible
 - no tiene por qué ser prominente
 - se determina a partir del entorno

Ejemplo de punto invisible



Se detecta el centro de la bola, a pesar de no tener una marca que lo destaque

1. segmentación de la bola verde por color
2. determinación de la circunferencia óptima
3. cálculo de su centro

Navegador quirúrgico

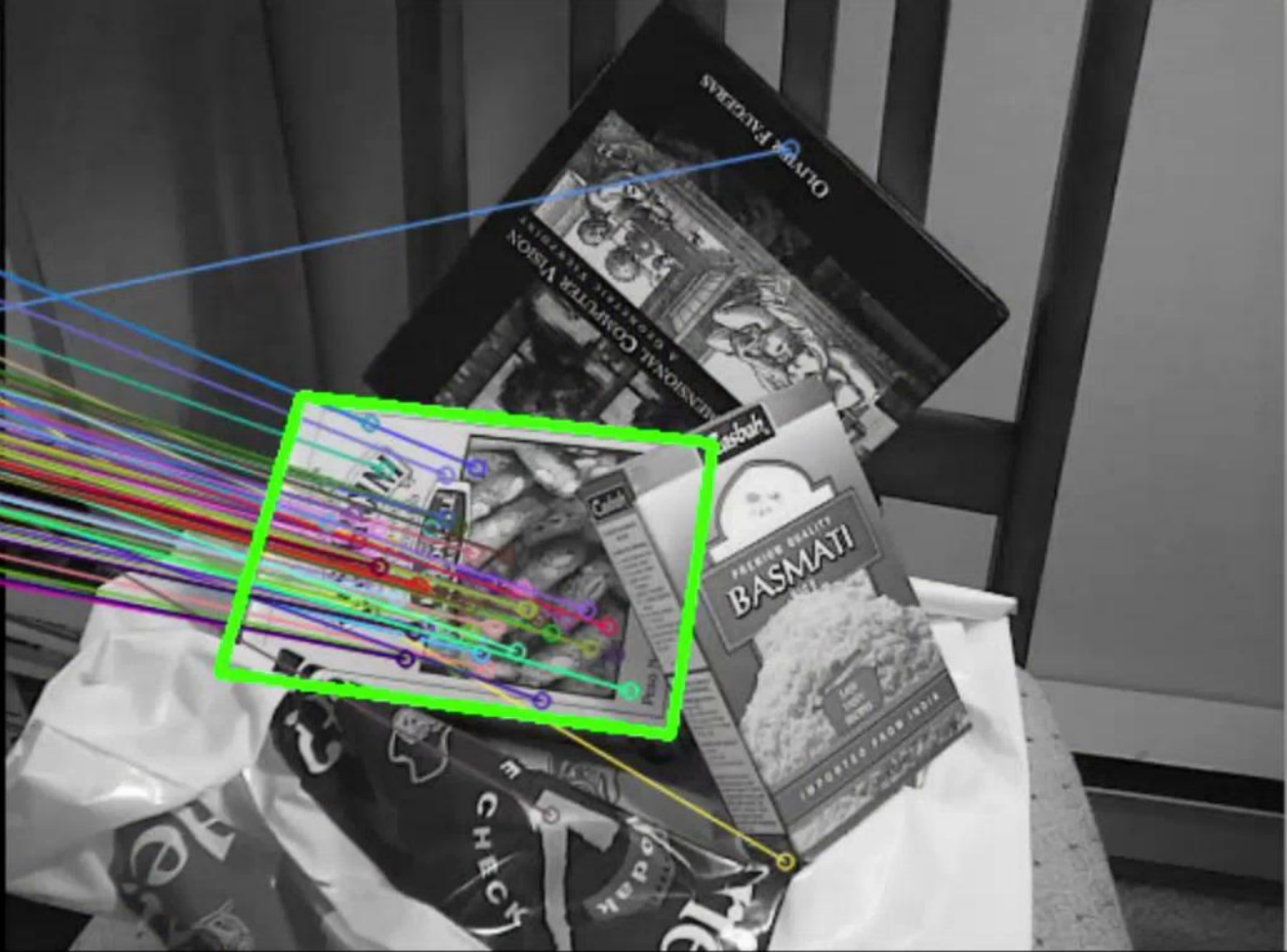
- El cirujano requiere conocer con precisión la posición de la punta del palpador
- Los cuatro leds de luz visible e infrarroja son **balizas** que serán detectadas por visión artificial
- A partir de su posición 2D en la imagen, el sistema calcula la posición 3D del palpador



Balizas

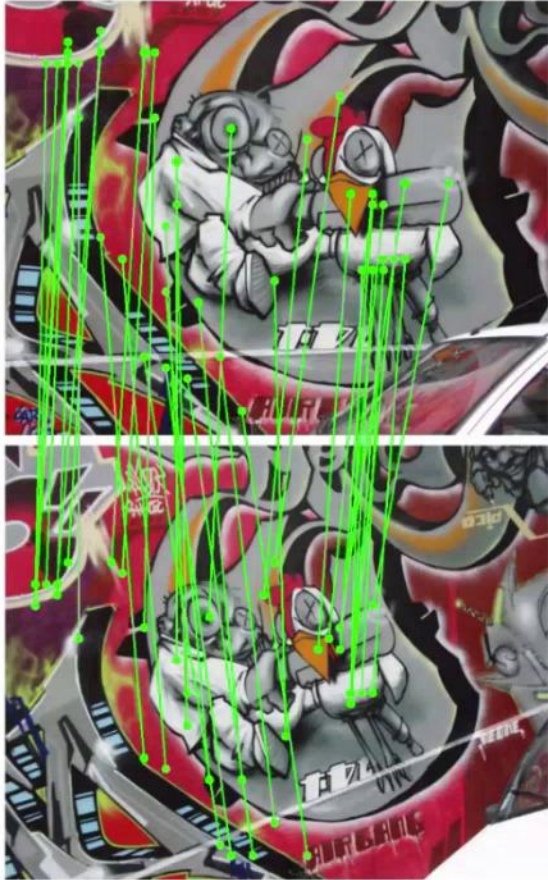


- No existe una función para detectar una baliza arbitraria
- Se debe desarrollar un pipeline a medida
- Se busca una solución de compromiso entre el detector y la baliza utilizada
- Son comunes
 - **balizas esféricas**, que se pueden detectar de la misma manera en todas direcciones
 - **balizas luminosas**, o de colores intensos, que destaquen en la imagen y sean fácilmente segmentables
 - incluídas las **balizas infrarrojas**, siempre que la cámara la vea
 - **balizas intermitentes**, codifican un identificador, y se pueden individualizar cuando se observan varias en la misma imagen

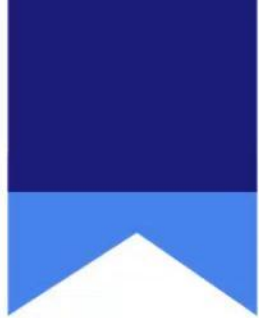


Pose
sin marcador

Marcador virtual



- Se utiliza una textura de referencia, usualmente plana, como marcador virtual
- Cada keypoint de referencia se convierte en un punto 3D
 - si la referencia es plana, la coordenada Z es cero
 - si la referencia no es plana, hay que asignar la coordenada Z correcta a cada keypoint
- Las correspondencias son numerosas y suelen contener outliers
 - se utiliza PnP robusto
 - `solvePnP Ransac()`



Referencias

- [cv::QRCodeDetector Class Reference](#)
- [cv::aruco::ArucoDetector Class Reference](#)
- [Camera Calibration and 3D Reconstruction Reference](#)
- [Example of pose estimation of calibration pattern](#)